

NeoCRM + Inventory Enterprise Platform

Architecture Design Document (ADD)

Prepared by: Principal Enterprise Solution Architect Function **Based on:** NeoCRM + Inventory SRS v1.0 (23 July 2026) and Automation Gap Analysis (CRM-11 – CRM-24) **Status:** Draft — Chapters 1–17 (Chapter 18: Recommendations pending) **Classification:** Confidential

Table of Contents

1. Executive Summary
2. Overall Product Architecture
3. Backend Architecture
4. Shared Infrastructure
5. Database Architecture
6. API Architecture
7. Integration Architecture
8. Security Architecture
9. Automation Architecture
10. AI Architecture
11. Deployment Architecture
12. Engineering Discussion Plan
13. Engineering Standards
14. Micro Execution Plan
15. Three Month Timeline
16. Team Allocation
17. Risks

(Chapter 18 — Recommendations — to be appended upon completion)

Chapter 1 — Executive Summary

1.1 Purpose

This document is the official Architecture Design Document (ADD) for the **NeoCRM + Inventory Enterprise Platform**, consolidating the approved Software Requirements Specification (SRS v1.0, 23 July 2026) and the subsequent Automation Gap Analysis (twelve previously uncovered requirements, CRM-11 through CRM-24) into a single, engineering-grade architectural reference. Its purpose is to give solution architects, backend and frontend engineers, QA, DevOps, and integration partners one authoritative source for *how* the system will be built, not merely *what* it must do. Where the SRS states requirements, this document states designs, trade-offs, and execution guidance.

1.2 Business Context

The organization currently operates its commercial function (sales, quoting, customer relationships) and its operational function (stock, quality, dispatch) as functionally and, in practice, data-disconnected activities. This produces three recurring business problems documented in the SRS Executive Summary and Vision sections:

- **Reconciliation gaps** between what Sales believes is available to sell and what Operations knows is physically on hand.
- **Double-commitment risk** — stock promised to more than one customer because no single authoritative ledger exists.
- **Loss of traceability** from a customer order back to the physical lot, batch, and Certificate of Analysis (COA) that fulfilled it.

The business case for the platform, as stated in the source SRS, rests on five outcomes: a single source of truth for stock; concurrency-safe reservations; full order-to-lot traceability; automation of the lead-to-dispatch chain; and a phased delivery that produces usable value (platform + CRM) before the harder operational and quality-control capabilities are built.

The subsequent gap analysis surfaced twelve additional requirements not present in SRS v1.0 — spanning pipeline hygiene, pricing governance, inbound email automation, lead quality control, and reporting — which this ADD treats as first-class scope, not addenda. **Architect Recommendation:** these twelve items should be formally issued as SRS v1.1 before Phase 1 development freezes, since several of them (win snapshot, duplicate detection, rule-based routing) affect core CRM data model decisions that are expensive to retrofit later.

1.3 Technical Context

The platform is architected as a single enterprise system composed of two cooperating domains — **NeoCRM** (sales) and a **unified Stock & Operations system** (Inventory, Quality Control, Dispatch) — sharing one PostgreSQL-backed transactional core and one append-only inventory ledger. The defining technical constraint, stated explicitly and non-negotiably in the SRS, is:

Operations modules (Inventory, QC, Dispatch) are the only writers to the inventory ledger.
NeoCRM is a read-only consumer of stock state.

This is not an implementation detail — it is the central architectural decision that shapes the data model, the API layer, the concurrency strategy, and the automation engine described in later chapters. A second constraint carried through the entire technical context is that **Oracle and Tally remain systems of record** for master data and financial valuation respectively; this platform does not replace them, it governs data flow to and from them.

The gap-analysis items add a third technical theme not present in SRS v1.0: **inbound unstructured data ingestion** — parsing emails, spreadsheets, and web-search results into structured CRM records (CRM-14, CRM-17, CRM-18, CRM-19, CRM-22). These items depend on the Shared Platform’s AI Services Layer (PLT-10), which SRS v1.0 marks as **Could** priority and Phase 3. This creates a scheduling tension addressed in Chapter 14 (Micro Execution Plan): several gap-analysis items cannot ship before PLT-10 exists, regardless of their own priority.

1.4 Architecture (High-Level System View)

flowchart TB

```
subgraph SALES["NeoCRM — Sales Domain"]
    CRM[CRM Core<br/>Leads · Quotes · Win Snapshot]
end

subgraph OPS["Stock & Operations Domain"]
    INV[Inventory]
    QC[Quality Control]
    DSP[Dispatch]
end

subgraph PLT["Shared Platform Services"]
    AUTH[Auth · RBAC]
    NOTIF[Notifications]
    AUDIT[Audit Log]
    DOC[Documents]
    DASH[Dashboards / Reporting]
    SEARCH[Global Search]
    AI[AI Services Layer]
end

LEDGER[(Single Inventory Ledger<br/>Append-only · Concurrency-safe)]
API[Unified API Layer]
```

```

DB[(PostgreSQL)]

subgraph EXT["External Systems"]
  ORACLE[Oracle - Master Data]
  TALLY[Tally - Valuation]
  COINCAP[Coincap]
  WA[WhatsApp]
  EMAIL[Email]
  GTM[GTM Google Sheet]
end

CRM -- reads only --> API
INV -- writes --> LEDGER
QC -- writes --> LEDGER
DSP -- writes --> LEDGER
API --> LEDGER
API --> DB
PLT --- API

API --> ORACLE
API --> TALLY
API --> COINCAP
API --> WA
API --> EMAIL
API --> GTM

```

This diagram reflects the SRS Figure 1 layered architecture, extended to show the GTM Sheet and inbound-email integration points introduced by the gap analysis (CRM-17, CRM-18, CRM-19), which this ADD treats as additional integration endpoints feeding the API layer rather than a separate ingestion subsystem.

1.5 Design Decisions

#	Decision	Source	Rationale
D1	Single append-only ledger, all Inventory/QC/Dispatch writes flow through it	SRS §3, §13.2	Eliminates reconciliation gaps by construction; enables full traceability
D2	CRM is strictly read-only against stock; never writes ledger entries directly	SRS §6, §3.1	Prevents sales-side double-commitment; enforces single-writer discipline
D3	Master data sourced from Oracle read-only; valuation posted to Tally outbound-only	SRS §10	Preserves existing systems of record; avoids scope creep into ERP replacement

D4	Win Snapshot introduced as a new immutable entity, distinct from mutable Quotation	Gap analysis CRM-12/13	Architect Recommendation: applies the ledger's append-only pattern to CRM state, keeping design language consistent platform-wide
D5	Price List versions are append-only with pending/approved states	Gap analysis CRM-14/15	Architect Recommendation: same immutable-versioning pattern reused a third time; should be documented once as a shared platform pattern (see Chapter 13)
D6	Unstructured email/web ingestion routed through AI Services Layer (PLT-10), not bespoke parsers	Gap analysis CRM-17/18/22	Architect Recommendation: avoids three independent NLP implementations; creates a scheduling dependency on Phase 3

1.6 Advantages

- A single ledger with one write-owner set (Inventory/QC/Dispatch) removes an entire class of reconciliation bugs by design rather than by process discipline.
- Phased delivery (Platform+CRM → Operations → AI/Automation) ships a usable, revenue-relevant system (quoting, pipeline) in Phase 1 without waiting for the harder ledger and concurrency work.
- Reuse of the “immutable, append-only, compensating-correction” pattern across the ledger, Win Snapshot, and Price List versions reduces architectural surface area and onboarding cost for new engineers.
- Read-only integration with Oracle and outbound-only integration with Tally minimizes blast radius of platform defects on existing financial and master-data systems of record.

1.7 Disadvantages

- The single-ledger, single-writer-set design concentrates significant complexity (concurrency safety, FEFO allocation, transactional automation) into the Inventory/QC/Dispatch modules delivered in Phase 2; Phase 1's CRM work will feel comparatively “easy,” which can create schedule-confidence risk heading into Phase 2.
- Strict CRM read-only-on-stock discipline requires near-real-time propagation (NFR-05) to avoid sales teams quoting against stale availability; this is a hard non-functional requirement, not a nice-to-have, and under-investment here directly undermines the platform's core value proposition.
- Deferring the AI Services Layer to Phase 3 (per SRS priority) while several gap-analysis items depend on it (CRM-17/18/22) means those items are structurally blocked, not merely

lower priority — this should be made explicit to stakeholders to avoid mis-set expectations.

1.8 Alternatives Considered

Alternative	Why Not Chosen
Separate ledgers per module (Inventory, QC, Dispatch each maintain own stock view)	Reintroduces the exact reconciliation-gap problem the platform exists to solve (SRS §1)
CRM writes directly to ledger on quote/win	Violates single-writer discipline; raises double-commitment risk under concurrent leads
Replace Oracle/Tally with platform-native master data and valuation	Explicitly out of scope per SRS §4.2; far larger program with different risk profile
Build bespoke NLP parsers per email-ingestion requirement instead of a shared AI layer	Triples implementation and maintenance cost for materially similar capability

1.9 Dependencies

- Phase 2 (Inventory/QC/Dispatch) is structurally dependent on Phase 1's shared platform and API layer (SRS §15.2).
- Dispatch's allocation logic depends on QC batch release state (DSP-03) — Dispatch cannot be meaningfully delivered ahead of QC.
- Automations (Section 11 of SRS, and AUT-07/08/09) depend on reliable event delivery across modules; this is a cross-cutting infrastructure dependency, not a feature of any one module.
- Gap-analysis pricing items (CRM-16) depend on CRM-14/15 existing first — the pre-check cannot function without a governed price list source.
- Gap-analysis email/enrichment items (CRM-17, CRM-18, CRM-22) depend on PLT-10 (AI Services Layer).

1.10 Future Scope

Per SRS §14 (Phase 3) and the gap analysis, future scope beyond the 40–100 page architecture covered here includes: advanced workflow automation and exception handling, cross-domain analytics, production hardening of all five external integrations, and expansion of AI assistance beyond drafting/summarization into predictive areas (e.g., demand forecasting feeding INV-12 reorder triggers) — **Architect Recommendation:** this last item is not currently in either source document and should be raised with the Product owner as a candidate for a future SRS revision, not assumed as in-scope.

1.11 Implementation Recommendations

- Formalize SRS v1.1 incorporating CRM-11 through CRM-24 before Phase 1 requirements freeze (Architect Recommendation).
- Document the “immutable versioned record” pattern once, centrally, and require all three consumers (Ledger, Win Snapshot, Price List) to implement it identically (Architect Recommendation) — detailed in Chapter 13.
- Treat NFR-04 (concurrency-safe reservations) and NFR-05 (near-real-time propagation) as Phase 2 architectural spikes, not late-stage hardening — both are load-bearing for the platform’s core value proposition.

1.12 Open Questions

- What is the acceptable latency bound for “near real-time” (NFR-05)? SRS does not quantify this; a target SLA (e.g., <2s p95) should be agreed before Phase 2 design.
- What confidence threshold governs auto-advancing a lead stage from PO extraction (CRM-18) versus routing to human review? Not specified in either source document.
- Is single sign-on (PLT-01, “where available”) a hard Phase 1 requirement or best-effort? Wording is ambiguous in the SRS.

1.13 Risks

Carried forward from SRS §15.3 (reservation race conditions, ledger corruption, integration outages, scope creep, stale CRM stock views), with one addition from this gap analysis: **AI-layer scheduling risk** — multiple gap-analysis items are silently blocked on Phase 3 AI infrastructure, creating a risk of stakeholders expecting Phase 2 delivery of capabilities that are structurally Phase 3-dependent. Full risk register is detailed in Chapter 17.

1.14 Best Practices

- Every new entity introduced beyond the original SRS (Win Snapshot, Price List Version) must be evaluated against the ledger’s own design principles (§13.2) — append-only, compensating corrections, full event linkage — before being approved for implementation.
 - RBAC (PLT-02) must gate every new capability introduced by the gap analysis (e.g., who may approve a price list, who may resolve a flagged duplicate) at the same action-level granularity mandated for the original SRS scope.
-

Chapter 2 — Overall Product Architecture

2.1 Purpose

This chapter defines the overall product architecture: how the platform is decomposed into domains and modules, how those modules are bounded and responsible for their own logic, and how the shared platform services and integration layer bind them together. It is the architectural expansion of SRS §3 (Solution Architecture Overview), §4 (Scope), and §13 (Data & Ledger Model), read together with the module boundaries implied by the gap-analysis workstreams.

2.2 Business Context

The business intent behind the product architecture is stated plainly in the SRS vision: *“Operations write to the ledger; CRM only reads.”* This is a business rule before it is a technical one — it exists to prevent Sales from ever promising stock that Operations has not actually committed to release. Every architectural boundary in this chapter exists in service of that rule. The gap-analysis items reinforce this business intent in two ways: the Win Snapshot (CRM-12/13) freezes commercial terms at the moment Sales creates a downstream obligation (a reservation via AUT-01), and the Pricing Pre-Check (CRM-16) prevents Sales from quoting a rate the business has not approved. Both are, at root, the same governance principle applied to commercial data that the ledger applies to physical stock.

2.3 Technical Context

The product is a modular monolith-oriented architecture (not a full microservices decomposition) unified by one PostgreSQL datastore and one API layer, per SRS §3: *“All modules persist to a shared PostgreSQL database and communicate through a common API layer.”* This is an explicit architectural choice in the source document, not an assumption introduced by this ADD. Module boundaries in this architecture are therefore **logical/domain boundaries enforced by API contracts and RBAC**, not physical service boundaries. **Architect Recommendation:** this should remain a modular monolith through Phase 2; splitting into physical microservices before the ledger’s concurrency model is proven in production would introduce distributed-transaction risk (see NFR-04) without a corresponding business benefit at current scale.

2.4 Architecture

2.4.1 Domain and Module Decomposition

```
flowchart TB
    subgraph D1["Domain: Sales (NeoCRM)"]
```

```

    M1[Customer & Lead Mgmt<br/>CRM-01..03, 20, 21]
    M2[Quotation & Samples<br/>CRM-04..06]
    M3[Win Capture<br/>CRM-10, 12, 13 - NEW]
    M4[Pricing Governance<br/>CRM-14..16 - NEW]
    M5[Insight & AI Assist<br/>CRM-07..09]
    M6[Inbound Lead Capture<br/>CRM-17..19, 22 - NEW]
end

subgraph D2["Domain: Stock & Operations"]
    M7[Inventory<br/>INV-01..12]
    M8[Quality Control<br/>QC-01..08]
    M9[Dispatch<br/>DSP-01..08]
end

subgraph D3["Domain: Shared Platform"]
    M10[Auth/RBAC/Users<br/>PLT-01..03]
    M11[Notifications<br/>PLT-04]
    M12[Audit<br/>PLT-05]
    M13[Documents<br/>PLT-06]
    M14[Dashboards/Reporting/Search<br/>PLT-07..09]
    M15[AI Services<br/>PLT-10]
    M16[API Layer<br/>PLT-11]
end

subgraph D4["Domain: Integration"]
    M17[Oracle · Tally · Coincap · WhatsApp · Email · GTM Sheet]
end

D1 -- read-only stock queries --> M16
D2 -- ledger writes --> M16
M16 --> D3
M16 --> D4

```

2.4.2 Module Responsibility Matrix

Module	Owns	Reads From	Writes To	New (Gap Analysis)
Customer & Lead Mgmt	Customers, leads, pipeline stages	—	Own tables	Extended: routing (CRM-20), dedup (CRM-21)
Quotation & Samples	Quotations, sample requests	Ledger (read-only, via API)	Own tables	—
Win	Win Snapshot		Own tables;	Yes (CRM-

Capture	(immutable)	Quotation	raises AUT-01 event	12/13)
Pricing Governance	Price List versions (pending/approved)	Email/upload ingestion	Own tables; gates Quotation send	Yes (CRM-14/15/16)
Insight & AI Assist	Dashboards, reports, AI drafting	Own + Ledger read	—	—
Inbound Lead Capture	Parsed leads/enrichment candidates	Email inbox, GTM Sheet, web search (via AI layer)	Feeds Lead Mgmt (post-dedup/routing)	Yes (CRM-17/18/19/22)
Inventory	Ledger, warehouses, lots, reservations	—	The ledger	—
Quality Control	Sample/test records, COA, batch release state	Inventory (lot data)	Ledger status transition, notifies Inventory (AUT-02)	—
Dispatch	Delivery orders, allocation, packing, export docs	QC status, Reservations	Ledger (reduces stock via AUT-03)	—
Shared Platform (all)	Cross-cutting concerns	—	—	—
Integration	External connectors	Oracle	Tally, WhatsApp, Email	GTM Sheet is new endpoint

2.4.3 Entity Relationship Overview (Conceptual)

```

erDiagram
    CUSTOMER ||--o{ LEAD : has
    LEAD ||--o| WIN_SNAPSHOT : "on win"
    LEAD ||--o{ QUOTATION : generates
    QUOTATION }o--|| PRICE_LIST_VERSION : "validated against"
    WIN_SNAPSHOT ||--o{ RESERVATION : triggers
    ITEM ||--o{ LOT : "has"
    LOT ||--o{ LEDGER_ENTRY : "referenced by"
    LOT ||--o| QUALITY_RECORD : "gated by"
    RESERVATION }o--|| LOT : holds
    DELIVERY_ORDER ||--o{ RESERVATION : consumes

```

```

DELIVERY_ORDER ||--o{ LEDGER_ENTRY : produces
WAREHOUSE ||--o{ LOCATION : contains
LOCATION ||--o{ CONTAINER : holds
CONTAINER }o--|| LOT : "groups qty of"

```

This conceptual model directly reflects SRS §13.1 (Core Entities) with two additions from the gap analysis: **WIN_SNAPSHOT** (CRM-12/13) sitting between LEAD and RESERVATION as the immutable record that triggers AUT-01, and **PRICE_LIST_VERSION** gating QUOTATION issuance (CRM-16). Both are drawn with the same relationship style as ledger entities to make the pattern-reuse explicit.

2.5 Design Decisions

#	Decision	Rationale
D7	Modular monolith on a single PostgreSQL instance, not microservices	Matches SRS §3 explicitly; avoids distributed-transaction complexity for ledger writes
D8	Inbound Lead Capture (CRM-17/18/19/22) modeled as a distinct module feeding the existing Lead Mgmt module, not a replacement pipeline	Keeps a single lead-creation path so routing (CRM-20) and dedup (CRM-21) apply uniformly regardless of source
D9	Pricing Governance modeled as its own module rather than folded into Quotation	Price lists have an independent approval lifecycle (pending → approved) distinct from any single quotation

2.6 Advantages

- A modular monolith with domain-bounded modules gives most of microservices' organizational clarity (clear ownership, testable boundaries) without the operational cost of distributed transactions across the ledger.
- Routing all new lead sources (email, GTM sheet, enrichment) through the existing Lead Mgmt module's dedup/routing logic (D8) avoids the classic integration bug of "three ways in, three different behaviors."

2.7 Disadvantages

- A single PostgreSQL instance is a scalability ceiling that will eventually need addressing (NFR-06 "Should" scales horizontally) — the modular monolith defers, but does not solve, that question.

- Centering Pricing Governance and Win Capture as new modules increases Phase 1/2 scope beyond the original SRS estimate; this should be reflected in re-baselined estimation (Chapter 15), not absorbed silently into existing phase timelines.

2.8 Alternatives Considered

Alternative	Why Not Chosen
Full microservices per module with independent datastores	Contradicts SRS §3's explicit shared-database design; introduces distributed ledger-consistency risk directly counter to NFR-02/04
Fold Win Snapshot into Quotation as a "locked" flag	Loses the clean append-only/immutable semantics the ledger pattern relies on; risks accidental mutation
Fold Pricing Governance into Shared Platform Documents module	Price lists are structured, versioned business data with an approval workflow, not a document attachment — mismatched semantics

2.9 Dependencies

- The Inbound Lead Capture module (M6) depends on the AI Services module (M15), which is Phase 3 per SRS §14 — this is the same dependency flagged in Chapter 1 and recurs structurally throughout this architecture.
- Pricing Governance (M4) must exist before the Pricing Pre-Check can be enforced in Quotation (M2).
- Win Capture (M3) depends on Quotation (M2) existing, and is itself a dependency of Inventory's AUT-01 reservation trigger.

2.10 Future Scope

As the platform matures past Phase 3, module boundaries described here (particularly Inventory/QC/Dispatch) are natural candidates for extraction into independently deployable services if transaction volume outgrows a single PostgreSQL instance — **Architect**

Recommendation: this is a Phase 4+ consideration and out of scope for the current 40–100 page architecture; it is noted here only so the module boundaries chosen now don't preclude it later.

2.11 Implementation Recommendations

- Enforce module boundaries at the code-repository/package level even within the monolith,

so a future extraction to services is a refactor, not a rewrite (Architect Recommendation).

- Require the Inbound Lead Capture module to always terminate in a call to Lead Mgmt's create-or-route function — never write directly to the leads table from an integration handler (Architect Recommendation, ties to D8).

2.12 Open Questions

- Does Pricing Governance require multi-level approval (e.g., regional Sales Head vs. global) or is single-approver (Sales Head) sufficient as stated in the gap analysis? Source material specifies only "Sales Head."
- Should Win Snapshot support any post-freeze correction path (e.g., agreed compensating amendment), or is it strictly immutable with a new Win Snapshot superseding the old one? Not specified.

2.13 Risks

- Scope growth from five new modules (Win Capture, Pricing Governance, Inbound Lead Capture, plus enrichment and reporting extensions) against a SRS v1.0 baseline risks Phase 1/2 estimate slippage if not explicitly re-baselined (links to SRS §15.3 "Scope creep across modules").

2.14 Best Practices

- Every module diagram and responsibility table in this ADD should be kept in sync with the code repository's actual package structure as implementation proceeds — architecture documentation that drifts from the codebase loses value faster than it is created.
- New modules introduced by the gap analysis should each get their own RBAC action set (per PLT-02) rather than inheriting permissions from a "parent" module, to keep access control auditable per NFR-01.

Chapter 3 — Backend Architecture

3.1 Purpose

This chapter specifies the backend architecture: service structure, transactional boundaries, concurrency handling for the ledger, the automation/event execution model, and the internal layering that implements the modules defined in Chapter 2. It expands SRS §3.1 (Component Responsibilities), §7.2 (Ledger & Movements), §11 (Business Process Automations), and NFR-01

through NFR-05.

3.2 Business Context

The backend exists to enforce, in code, the business guarantees the SRS promises: that stock is never double-committed (O2), that every movement is traceable to an actor and event (O4), and that regulated quality gates are respected before dispatch (O5). The gap-analysis items add two further backend-enforced guarantees: that a won deal's commercial terms cannot silently change after the fact (CRM-12/13), and that a quote cannot leave the building at an unapproved rate (CRM-16). These are business trust guarantees before they are engineering concerns — a bug here is a customer dispute or a financial loss, not just a defect.

3.3 Technical Context

The backend is a layered, module-organized service sitting behind the Unified API Layer (PLT-11), operating against a single PostgreSQL database (PLT-12). The single hardest technical requirement it must satisfy is **NFR-04: concurrency-safe reservations — no lot may be reserved beyond its available quantity under load**. This governs the transactional design of the entire Inventory write path and, by extension, of AUT-01 and AUT-03. **Architect Recommendation:** this requirement should be treated as the backend's primary design constraint, ahead of convenience or velocity concerns, because a violation directly contradicts the platform's core value proposition (O2) stated in Chapter 1.

3.4 Architecture

3.4.1 Layered Backend Structure

```
flowchart TB
    subgraph L1["API Layer (PLT-11)"]
        REST[REST/API Endpoints]
    end
    subgraph L2["Application/Service Layer"]
        SVC_CRM[CRM Services]
        SVC_INV[Inventory Services]
        SVC_QC[QC Services]
        SVC_DSP[Dispatch Services]
        SVC_AUT[Automation Engine]
    end
    subgraph L3["Domain Layer"]
        DOM_LEDGER[Ledger Domain Logic<br/>FEFO · Reservation · Adjustment]
        DOM_WIN[Win Snapshot Domain Logic]
        DOM_PRICE[Price List Domain Logic]
    end
    subgraph L4["Data Access Layer"]
```

```

    REPO[Repositories / ORM]
end
subgraph L5["Data Store"]
    PG[(PostgreSQL)]
end

REST --> L2
SVC_CRM --> DOM_WIN
SVC_CRM --> DOM_PRICE
SVC_INV --> DOM_LEDGER
SVC_QC --> DOM_LEDGER
SVC_DSP --> DOM_LEDGER
SVC_AUT --> SVC_INV
SVC_AUT --> SVC_QC
SVC_AUT --> SVC_DSP
L3 --> REPO
REPO --> PG

```

3.4.2 Concurrency-Safe Reservation Flow (NFR-04, INV-05, INV-06)

```

sequenceDiagram
    participant CRM as CRM (Win event)
    participant AUT as Automation Engine (AUT-01)
    participant INVSvc as Inventory Service
    participant DB as PostgreSQL (Ledger)

    CRM->>AUT: Lead Won (Win Snapshot payload)
    AUT->>INVSvc: Request reservation(item, qty)
    INVSvc->>DB: BEGIN TRANSACTION
    INVSvc->>DB: SELECT lots FOR UPDATE (FEFO order, row-level lock)
    DB-->>INVSvc: Locked candidate lots
    INVSvc->>INVSvc: Compute available = on_hand - reserved - quarantined
    alt sufficient available quantity
        INVSvc->>DB: INSERT reservation + ledger entry (reserved)
        INVSvc->>DB: COMMIT
        INVSvc-->>AUT: Reservation confirmed
    else insufficient quantity
        INVSvc->>DB: ROLLBACK
        INVSvc-->>AUT: Reservation failed
        AUT-->>CRM: Notify failure (route to operator queue, AUT-09)
    end
end

```

This sequence directly implements SRS INV-05 (“concurrency-safe reservations that hold stock against a source event without double-committing”) using row-level locking (`SELECT ... FOR UPDATE`) inside a single transaction as the mechanism. **Architect Recommendation:** this is the recommended implementation pattern — the SRS states the requirement (concurrency-safety)

but does not mandate a mechanism; row-level locking within PostgreSQL is chosen here over application-level distributed locks or optimistic concurrency with retry, because it keeps the guarantee inside the same transaction as the ledger write, minimizing the window for race conditions (SRS Risk: “Reservation race conditions,” §15.3).

3.4.3 Automation Engine — Transactional Execution Model (AUT-07)

AUT-07 requires that “*automations execute transactionally; partial effects must not corrupt the ledger.*” The backend implements this as an **event-sourced-trigger, transactionally-bound-effect** pattern:

```
flowchart LR
    EVT[Business Event<br/>e.g. Lead Won, QC Release, Dispatch Executed] --> Q[Automa
    Q --> HANDLER[Automation Handler]
    HANDLER --> TXN{Single DB Transaction}
    TXN -->|success| EFFECT[Ledger/State Effect + Audit Entry]
    TXN -->|failure| RETRY[Operator Retry Queue<br/>AUT-09]
    EFFECT --> AUDIT[Audit Log - AUT-08]
```

Each automation (AUT-01 through AUT-06) is implemented as a handler that either fully commits its ledger/state effect and its audit entry in one transaction, or fully rolls back and surfaces to the AUT-09 operator queue. No automation handler is permitted to perform a partial write followed by a “best-effort” cleanup step — this directly satisfies AUT-07 and AUT-08 (attributability).

3.4.4 Win Snapshot & Pricing Governance Backend Logic (New)

Requirement	Backend Behavior
CRM-12 (Win Snapshot)	On lead-won event, service layer copies current Quotation state into a new <code>WinSnapshot</code> row, sets an immutability flag, generates PDF via PLT-06 (Documents), and never permits UPDATE on that row thereafter — only new compensating snapshots referencing the original.
CRM-13 (Revenue/tonnage calc)	Computed at Win Snapshot creation time from frozen line items, persisted alongside the snapshot (not recalculated later from mutable Quotation data) so reporting (CRM-23, CRM-24) reads a stable figure.
CRM-14/15 (Price List ingestion/approval)	Ingested file (email attachment or upload) parsed into a <code>PriceListVersion</code> row in <code>pending</code> state; approval action by Sales Head is a state transition (<code>pending</code> → <code>approved</code>), previous approved version moved to <code>archived</code> , never deleted — mirrors ledger’s append-only principle.
CRM-16 (Pricing	Synchronous check in the Quotation send path: looks up latest <code>approved</code>

pre-check)	PriceListVersion, compares line rate, blocks/warns per configured tolerance before the send action commits.
------------	---

3.5 Design Decisions

#	Decision	Rationale
D10	Row-level locking (<code>SELECT FOR UPDATE</code>) inside a single transaction for reservations	Satisfies NFR-04 with the strongest consistency guarantee available in PostgreSQL without external coordination
D11	Automation handlers are all-or-nothing within one DB transaction; failures go to a retry queue, never partial-commit	Directly satisfies AUT-07; makes AUT-09's "failed automations surface to operator queue" implementable
D12	Win Snapshot revenue/tonnage computed once, at freeze time, and stored — not recomputed on read	Guarantees reporting stability even if pricing rules or Quotation data model change later

3.6 Advantages

- Row-level locking keeps the concurrency guarantee inside PostgreSQL's own transactional machinery, avoiding the operational complexity of a separate distributed lock service.
- The uniform "event → transactional handler → effect or retry queue" pattern (3.4.3) means AUT-01 through AUT-06 all share one execution model, one failure-handling path, and one audit mechanism — reducing the number of distinct code paths QA must certify.

3.7 Disadvantages

- Row-level locking under high concurrent load on hot lots (e.g., a fast-moving SKU with many simultaneous leads winning) can create lock contention; this is a known trade-off of pessimistic locking and should be load-tested explicitly against realistic peak concurrency before Phase 2 sign-off.
- Storing computed revenue/tonnage at freeze time (D12) means any pricing *methodology* bug discovered after the fact requires a compensating snapshot correction process, not a simple recalculation — this is intentional (matches ledger philosophy) but has real operational cost if it occurs.

3.8 Alternatives Considered

Alternative	Why Not Chosen
-------------	----------------

Optimistic concurrency (version column + retry on conflict) for reservations	Under high contention on popular SKUs, retry storms could degrade UX at exactly the moment (end of quarter, high lead-win volume) reliability matters most
External distributed lock service (e.g., Redis-based) for reservations	Adds an operational dependency and a second source of truth for locking state, increasing failure modes without a clear benefit given single-PostgreSQL-instance design (D7)
Recompute Win Snapshot revenue/tonnage on every report read	Simpler initially, but couples reporting correctness to Quotation/pricing schema stability indefinitely — fragile long-term

3.9 Dependencies

- The Automation Engine (3.4.3) is a prerequisite for AUT-01 through AUT-06 and therefore for INV-05, QC-08, and DSP-06 functioning end-to-end.
- CRM-16 (pricing pre-check) is backend-dependent on CRM-14/15's PriceListVersion table existing and being populated.
- CRM-13's stored revenue/tonnage figures are a backend dependency for CRM-23 (target actuals) and CRM-24 (effort reporting) in Chapter 9/reporting flows.

3.10 Future Scope

As transaction volume grows, the reservation locking strategy (D10) may need to evolve toward partitioned/sharded lot locking or queue-based serialization per warehouse to reduce contention — **Architect Recommendation:** this is a Phase 4+ scaling concern, not required for the current phased scope, and should be revisited only if load testing in Phase 2 reveals actual contention rather than pre-optimizing now.

3.11 Implementation Recommendations

- Load-test the reservation transaction (3.4.2) under realistic concurrent lead-win scenarios before Phase 2 exit criteria are signed off (Architect Recommendation).
- Implement the Automation Engine (3.4.3) as a single shared internal framework used by all six automations (AUT-01–06), not six independent implementations, to keep AUT-07/08 guarantees uniform (Architect Recommendation).

3.12 Open Questions

- What is the defined tolerance for the pricing pre-check (CRM-16) — exact match only, or a

permitted deviation percentage requiring override approval? Not specified in gap analysis.

- Should failed reservation attempts (3.4.2, insufficient quantity branch) automatically notify the Sales rep, or only surface via the AUT-09 operator queue? SRS/gap analysis do not specify a customer/sales-facing failure path.

3.13 Risks

- Lock contention under peak load (§3.7) is a new, backend-specific instantiation of the SRS's general "Reservation race conditions" risk (§15.3) and should be tracked as its own line item in the Phase 2 test plan.
- Compensating-snapshot correction process for Win Snapshot (§3.7) is currently undefined; absence of a defined correction path is itself an operational risk if a pricing error is discovered post-freeze.

3.14 Best Practices

- All ledger-affecting transactions must acquire locks in a consistent order (e.g., always lot-then-reservation) to avoid deadlocks under concurrent access — standard practice for row-level-locking systems, worth codifying explicitly in engineering standards (Chapter 13).
- Every automation handler should log both its trigger event ID and its resulting effect ID in the same audit entry, satisfying AUT-08's attributability requirement without relying on log correlation across separate systems.

Chapter 4 — Shared Infrastructure

4.1 Purpose

This chapter details the Shared Platform Services layer (SRS §5, PLT-01 through PLT-12) — the cross-cutting capabilities consumed by every domain module. It specifies how authentication, RBAC, notifications, audit, documents, dashboards/reporting/search, and the AI services layer are structured so that Chapters 2 and 3's module boundaries can rely on them as stable, shared infrastructure rather than each module reimplementing its own.

4.2 Business Context

Shared infrastructure exists so that trust guarantees — who can do what (RBAC), what happened and when (Audit), how people are informed (Notifications) — are enforced identically everywhere in the platform, rather than varying module by module. This matters commercially as

much as technically: NFR-01 (auditability) and NFR-03 (role security, deny-by-default) are the guarantees that let the business tell an auditor, a regulator, or a customer exactly who did what and why, across both the sales and operations sides of the business in one consistent answer.

4.3 Technical Context

Per SRS §3, shared services sit beside the domain modules and are accessed through the same Unified API Layer (PLT-11). They are *shared services*, not *shared databases per module* — all persist to the same PostgreSQL instance (PLT-12) established in Chapter 2. The gap analysis introduces new consumers of two of these services in particular: the Notification service (PLT-04) gains a new trigger source (stale-lead flagging, CRM-11), and the AI Services layer (PLT-10) gains four new consumers (CRM-17, 18, 19, 22) that did not exist in SRS v1.0's scope for that service.

4.4 Architecture

4.4.1 Shared Services Map

```
flowchart TB
    subgraph CONSUMERS["Domain Modules"]
        CRM[NeoCRM]
        INVQCD[Inventory / QC / Dispatch]
    end

    subgraph SHARED["Shared Platform Services"]
        AUTH[PLT-01 Auth + SSO]
        RBAC[PLT-02 RBAC]
        USERS[PLT-03 User/Org Mgmt]
        NOTIF[PLT-04 Notifications<br/>in-app · email · WhatsApp]
        AUDIT[PLT-05 Audit Log]
        DOCS[PLT-06 Document Mgmt]
        DASH[PLT-07 Dashboards]
        REPORT[PLT-08 Reporting Engine]
        SEARCH[PLT-09 Global Search]
        AI[PLT-10 AI Services]
    end

    API[PLT-11 Unified API Layer]
    DB[(PLT-12 PostgreSQL)]

    CONSUMERS --> API
    API --> SHARED
    SHARED --> DB
    AUTH --> RBAC
    RBAC --> USERS
```

4.4.2 RBAC Enforcement Pattern (PLT-02, NFR-03)

Every API call passes through a single authorization checkpoint before reaching module logic:

```
sequenceDiagram
    participant Client
    participant API as API Layer (PLT-11)
    participant RBAC as RBAC Service (PLT-02)
    participant AUDIT as Audit Log (PLT-05)
    participant MOD as Module Logic

    Client->>API: Request (action, entity)
    API->>RBAC: Check permission(user, role, action, entity)
    alt permitted
        RBAC-->>API: Allow
        API->>MOD: Execute action
        MOD-->>API: Result
        API->>AUDIT: Log(actor, action, entity, before/after, timestamp)
        API-->>Client: Response
    else denied
        RBAC-->>API: Deny (default)
        API-->>Client: 403 Forbidden
        API->>AUDIT: Log denied attempt
    end
end
```

This reflects NFR-03 verbatim (“access is denied by default and granted explicitly through RBAC”) and PLT-05’s requirement to capture actor, action, entity, before/after state, and timestamp on every business-significant action — including, per this design, denied attempts, which the SRS does not explicitly require but which is good practice for security observability (NFR-09).

4.4.3 Notification Service — New Trigger Sources

Trigger	Existing / New	Channel(s)	Source Requirement
Key state changes (lead stage, dispatch status)	Existing	In-app, email, WhatsApp	AUT-06
Low-stock / reorder	Existing	In-app, email	AUT-04
Approaching expiry	Existing	In-app, email	AUT-05
Stale-lead flag	New	In-app (dashboard highlight) + notification to owner/manager	CRM-11 (gap analysis)

Price list pending approval	New	In-app + email to Sales Head	CRM-15 (gap analysis)
Duplicate lead flagged for merge	New	In-app to Sales Head	CRM-21 (gap analysis)

Architect Recommendation: all new notification triggers introduced by the gap analysis should be implemented as new *event types* published to the existing PLT-04 notification service and its unified template set, not as bespoke notification code inside CRM-11/15/21 — this keeps the “unified template set” requirement (PLT-04) actually unified as new triggers accumulate.

4.4.4 AI Services Layer — Consumer Map (PLT-10, Priority: Could, Phase 3)

flowchart LR

```

AI[AI Services Layer — PLT-10]
AI --> C1[CRM-09: Email drafting,<br/>account summarization]
AI --> C2[CRM-17: Email → Lead parsing]
AI --> C3[CRM-18: Subject routing,<br/>PO extraction/matching]
AI --> C4[CRM-22: Web-search<br/>company enrichment]

```

This consumer map is the clearest illustration of the scheduling tension flagged in Chapters 1–2: **four separate requirements across two documents converge on one Phase 3, “Could”-priority shared service.** No amount of individual-requirement prioritization changes this — PLT-10 is a hard prerequisite for all four.

4.5 Design Decisions

#	Decision	Rationale
D13	RBAC checkpoint sits at the API layer, not duplicated per module	Single enforcement point makes NFR-03 auditable and testable once, not per module
D14	All new gap-analysis notification triggers route through existing PLT-04 as new event types	Preserves “unified template set” requirement; avoids notification-logic sprawl
D15	Denied RBAC attempts are also audit-logged	Architect Recommendation beyond stated SRS text — supports NFR-09 observability and security incident investigation

4.6 Advantages

- Centralizing RBAC at the API layer (D13) means every new module added in future phases inherits deny-by-default security automatically, with no risk of a module “forgetting” to

check permissions.

- Routing all new notification triggers through the existing service (D14) means the eventual addition of a new channel (e.g., SMS) only needs to be built once, not once per trigger type.

4.7 Disadvantages

- Centralizing AI consumption for four distinct requirements onto one Phase 3 service (4.4.4) means any delay or underinvestment in PLT-10 delays four gap-analysis requirements simultaneously — this concentration of risk should be visible to program management, not buried in individual ticket estimates.
- Audit-logging denied RBAC attempts (D15) at scale can generate significant log volume if a misconfigured client retries aggressively; retention/volume policy should be defined (see Chapter 8, Security Architecture).

4.8 Alternatives Considered

Alternative	Why Not Chosen
Per-module RBAC checks instead of a central API-layer checkpoint	Risks inconsistent enforcement and makes NFR-03 compliance harder to verify centrally
Bespoke notification logic per new gap-analysis trigger	Fragments the “unified template set” requirement (PLT-04) that the SRS explicitly calls for
Separate, dedicated AI service instances per consuming requirement (CRM-17 vs CRM-18 vs CRM-22)	Increases operational surface area for materially similar capability (structured extraction from unstructured text) without clear benefit

4.9 Dependencies

- Every domain module (Chapter 2) depends on Auth/RBAC (PLT-01/02) and Audit (PLT-05) being available from day one of Phase 1 — these are not optional infrastructure, they are load-bearing for every other requirement’s compliance story.
- CRM-11 (stale-lead flagging) depends on PLT-04 and PLT-07 (dashboards) both being in place — this is consistent with placing CRM-11 in Phase 1 per Chapter 1’s sequencing table.
- CRM-17/18/19/22 depend on PLT-10, which per SRS §14 is Phase 3.

4.10 Future Scope

Beyond Phase 3, the AI Services layer is a natural candidate for expansion into predictive

capabilities noted in Chapter 1 (§1.10) — **Architect Recommendation:** any such expansion should be scoped as a new SRS revision with its own requirements, not silently absorbed into PLT-10's existing "Could" priority scope.

4.11 Implementation Recommendations

- Build PLT-10 with a stable internal interface (e.g., "extract structured entity from unstructured text, with confidence score") early enough in Phase 2 planning that CRM-17/18/22 can be estimated against a real interface rather than a placeholder (Architect Recommendation).
- Instrument the RBAC checkpoint (4.4.2) with metrics (permit/deny counts per role/action) from day one to support both NFR-09 observability and future access-pattern-driven role refinement (Architect Recommendation).

4.12 Open Questions

- What SLA or accuracy threshold is expected of PLT-10's outputs before CRM-18's PO-extraction is trusted to auto-advance a stage versus route to review (echoes the open question raised in Chapter 3, §3.12)?
- Is single sign-on (PLT-01) required to integrate with a specific existing identity provider, or is provider choice open? Not specified in source documents.

4.13 Risks

- Concentration risk on PLT-10 (§4.7) is a new risk not present in the original SRS risk register (§15.3) and should be added explicitly to the consolidated risk register in Chapter 17.
- Audit log volume growth from denied-attempt logging (D15) should be monitored; unbounded growth without a retention policy risks conflicting with NFR-07 (backup/recovery) performance.

4.14 Best Practices

- Treat RBAC role/action definitions as versioned configuration, reviewed alongside code changes — not as ad hoc database rows edited outside of change control, given their direct role in NFR-01/NFR-03 compliance.
 - Every new notification event type (4.4.3) should be added to the unified template set with a defined owner (module) and defined recipient-resolution logic, documented in one central registry rather than scattered across module code.
-

Chapter 5 — Database Architecture

5.1 Purpose

This chapter specifies the database architecture: the physical schema design implementing the conceptual entity model from Chapter 2 (§2.4.3), the ledger’s storage-level guarantees, indexing and integrity strategy, and the schema additions required by the twelve gap-analysis requirements. It expands SRS §13 (Data & Ledger Model) and NFR-02 (immutable ledger), NFR-08 (data integrity).

5.2 Business Context

The database is where the platform’s most important business promise is literally enforced: that a stock movement, once recorded, cannot be silently altered, and that a commercial commitment (Win Snapshot), once frozen, cannot quietly change. Both a warehouse auditor and a finance controller need to be able to trust that what the database says happened is what actually happened — this is the operational meaning of NFR-01 (auditability) and NFR-02 (immutable ledger) at the storage layer.

5.3 Technical Context

The platform uses a single PostgreSQL instance (PLT-12) as the shared transactional datastore for all modules, per SRS §3 and §5. This chapter assumes PostgreSQL’s native capabilities — row-level locking (used in Chapter 3), foreign key constraints, check constraints, and transactional DDL — as the mechanisms for the business-rule integrity required by NFR-08.

Architect Recommendation: append-only tables (ledger, Win Snapshot, Price List versions) should additionally be protected at the database privilege level (REVOKE UPDATE/DELETE for the application role, only INSERT permitted) rather than relying solely on application-layer discipline — this converts an application convention into a database-enforced guarantee.

5.4 Architecture

5.4.1 Core Schema Overview

```
erDiagram
    ITEM ||--o{ LOT : "has"
    ITEM ||--o{ PRICE_LIST_LINE : "priced in"
    WAREHOUSE ||--o{ LOCATION : contains
    LOCATION ||--o{ CONTAINER : holds
    CONTAINER }o--|| LOT : "groups"
    LOT ||--o{ LEDGER_ENTRY : "referenced by"
    LOT ||--o{ QUALITY_RECORD : "gated by"
```

```

LOT ||--o{ RESERVATION : "held against"

CUSTOMER ||--o{ LEAD : has
LEAD ||--o{ QUOTATION : generates
LEAD ||--o{ WIN_SNAPSHOT : "on win"
QUOTATION }o--|| PRICE_LIST_VERSION : "validated against"
WIN_SNAPSHOT ||--o{ RESERVATION : triggers
RESERVATION }o--|| DELIVERY_ORDER : "consumed by"
DELIVERY_ORDER ||--o{ LEDGER_ENTRY : produces

PRICE_LIST_VERSION ||--o{ PRICE_LIST_LINE : contains

LEAD ||--o{ AUDIT_LOG : "generates"
LEDGER_ENTRY ||--o{ AUDIT_LOG : "generates"

```

5.4.2 Ledger Table Design (INV-04, NFR-02)

Column	Type	Constraint	Notes
ledger_entry_id	UUID	PK	—
entry_type	ENUM	NOT NULL	receipt / issue / transfer / adjustment / reservation / release
item_id	FK → item	NOT NULL	—
lot_id	FK → lot	NOT NULL	—
location_id	FK → location	NOT NULL	—
quantity	NUMERIC	NOT NULL, CHECK (quantity <> 0)	signed; positive = increase, negative = decrease
source_event_id	UUID	NOT NULL	links to originating business event (AUT-08 traceability)
reason_code	TEXT	NOT NULL for adjustments (INV-08)	—
created_by	FK → user	NOT NULL	actor for audit (PLT-05)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	append-only, never updated

No `updated_at` column exists on this table by design — its absence is itself an architectural control: there is nothing to update. Corrections are new rows referencing the original via

`source_event_id` , per SRS §13.2 (“adjustments are new, signed entries with reason codes”).

5.4.3 New Entities — Win Snapshot and Price List Version

WIN_SNAPSHOT (CRM-12, CRM-13)

Column	Type	Constraint	Notes
<code>win_snapshot_id</code>	UUID	PK	—
<code>lead_id</code>	FK → lead	NOT NULL	—
<code>quotation_id</code>	FK → quotation	NOT NULL	source at freeze time
rate, quantity, terms, tax, total	NUMERIC/TEXT	NOT NULL	frozen commercial terms
<code>revenue_amount</code>	NUMERIC	NOT NULL	computed at freeze (CRM-13)
<code>tonnage</code>	NUMERIC	NOT NULL	computed at freeze (CRM-13)
<code>pdf_document_id</code>	FK → document (PLT-06)	NOT NULL	generated at freeze
<code>superseded_by</code>	FK → <code>win_snapshot</code> , nullable	—	supports compensating-correction pattern (Ch.3 §3.12 open question)
<code>created_at</code>	TIMESTAMPTZ	NOT NULL	append-only

PRICE_LIST_VERSION (CRM-14, CRM-15)

Column	Type	Constraint	Notes
<code>price_list_version_id</code>	UUID	PK	—
<code>status</code>	ENUM	NOT NULL	pending / approved / archived
<code>source</code>	ENUM	NOT NULL	email_ingested / manual_upload
<code>submitted_by</code>	FK → user	NOT NULL	—
<code>approved_by</code>	FK → user, nullable	—	set on approval (CRM-15)
<code>approved_at</code>	TIMESTAMPTZ, nullable	—	—

created_at	TIMESTAMPTZ	NOT NULL	—
------------	-------------	----------	---

PRICE_LIST_LINE (child table): price_list_version_id, item_id, rate, unit — one row per item/rate pair, versioned together as a unit.

5.4.4 Indexing Strategy (Supports NFR-05 near-real-time reads)

Table	Index	Purpose
ledger_entry	(lot_id, created_at)	FEFO allocation queries, lot history
ledger_entry	(source_event_id)	Traceability lookups (AUT-08)
reservation	(lot_id, status)	Available-quantity computation (§13.2 formula)
lead	(stage, stage_entered_at)	Stale-lead flagging query (CRM-11)
price_list_version	(status, item_id via join)	Latest-approved lookup for pricing pre-check (CRM-16)
quality_record	(lot_id, status)	Dispatch allocation blocking (DSP-03)

Architect Recommendation: the (lead.stage, lead.stage_entered_at) composite index is specifically sized for CRM-11’s scheduled stale-lead scan (Chapter 4, §4.4.3) — without it, that job degrades to a full table scan as lead volume grows.

5.4.5 Available Quantity — Computed, Not Stored

Per SRS §13.2: *“Available quantity = on-hand – reserved – quarantined, computed from ledger state.”* This is implemented as a database view or computed query, **not** a denormalized stored column, to avoid the classic bug class of a cached balance drifting from its ledger source of truth:

```
-- Conceptual view, not literal production DDL
CREATE VIEW lot_available_quantity AS
SELECT lot_id,
       SUM(CASE WHEN entry_type IN ('receipt','release') THEN quantity ELSE 0 END)
       - SUM(CASE WHEN entry_type IN ('issue','reservation') THEN ABS(quantity) ELSE 0
       - COALESCE(quarantined_qty, 0) AS available_quantity
FROM ledger_entry
GROUP BY lot_id;
```

5.5 Design Decisions

#	Decision	Rationale
D16	Ledger table has no <code>updated_at</code> /no UPDATE grant — corrections are new rows	Enforces NFR-02 at the schema/privilege level, not just convention
D17	Available quantity computed via view, never denormalized/cached	Prevents balance drift; single source of truth remains the ledger
D18	Win Snapshot and Price List Version follow identical append-only + status-transition pattern as the ledger	Consistent with Chapter 1/2's "reuse the pattern" recommendation
D19	<code>superseded_by</code> self-reference on Win Snapshot	Answers Chapter 3's open question on post-freeze correction: a new snapshot supersedes, original is never mutated

5.6 Advantages

- Enforcing append-only-ness at the database privilege level (D16) means even a future application bug cannot silently violate NFR-02 — the database itself refuses the UPDATE.
- Computing available quantity via view (D17) guarantees the number shown in a CRM quote (CRM-05) is always derived from the current ledger state, directly serving NFR-05's near-real-time requirement without a separate cache-invalidation mechanism to get wrong.

5.7 Disadvantages

- Computing available quantity via a view/aggregate query rather than a cached column can become a read-performance concern at high ledger-entry volume per lot; this trade-off should be monitored and may require a materialized view with a defined refresh strategy as data grows (see Future Scope).
- The `superseded_by` self-reference on Win Snapshot (D19) introduces a chain-following read pattern ("find the current version of this snapshot") that reporting queries (CRM-23/24) must handle correctly — a source of potential bugs if not centralized in one query helper.

5.8 Alternatives Considered

Alternative	Why Not Chosen
Denormalized <code>available_qty</code> column on Lot, updated by trigger	Reintroduces a cache-consistency risk the ledger design exists to eliminate

Soft-delete/versioned rows with an <code>is_current</code> flag for the ledger	More error-prone than pure append-only; risks accidental mutation of "current" flag under concurrency
Store Win Snapshot corrections as a full mutable edit with a change log table	Breaks the immutability guarantee at its core; change log becomes advisory rather than structurally enforced

5.9 Dependencies

- The reservation transaction design in Chapter 3 (§3.4.2) depends on the `(lot_id, status)` index (§5.4.4) existing for its `FOR UPDATE` query to perform acceptably under load.
- CRM-11's stale-lead job depends on the composite index in §5.4.4.
- CRM-16's pricing pre-check depends on an efficient "latest approved version for item X" query, which depends on the `(status, item_id)` index.

5.10 Future Scope

If ledger-entry volume per lot grows large enough that the computed-view approach (§5.4.5, D17) becomes a measurable read-latency problem, a materialized view with a scheduled or event-triggered refresh is the natural evolution — **Architect Recommendation:** do not pre-build this; instrument query latency in Phase 2 production and only introduce materialization if data shows it's needed, to avoid adding cache-consistency complexity prematurely.

5.11 Implementation Recommendations

- Apply database-level privilege restrictions (`REVOKE UPDATE/DELETE`) to `ledger_entry`, `win_snapshot`, and `price_list_version` tables as part of initial schema migration, not as a later hardening pass (Architect Recommendation).
- Centralize the "resolve current Win Snapshot version" logic (following `superseded_by` chains) in one repository function used by every reporting consumer, rather than letting each report re-implement the chain-following logic (Architect Recommendation).

5.12 Open Questions

- What data retention policy applies to archived Price List versions and superseded Win Snapshots — indefinite retention, or a defined archival/purge policy after N years? Not specified in source documents; likely governed by external compliance requirements not captured in either SRS or gap analysis.
- Should quarantined quantity (§5.4.5 formula) be tracked as its own ledger entry type, or as a

separate table joined into the view? Current SRS text implies it's part of the ledger state but doesn't specify the storage mechanism.

5.13 Risks

- Read performance of the computed available-quantity view (§5.7) is an unquantified risk until load-tested — should be added to the Phase 2 test plan explicitly.
- Undefined retention policy (§5.12) is a compliance risk if the platform is later subject to a formal audit or legal hold requirement that source documents do not anticipate.

5.14 Best Practices

- Every new append-only entity introduced in future phases should be reviewed against the same checklist used for the ledger (no UPDATE grant, `source_event_id` /equivalent linkage, `created_at` only) before being approved — codify this checklist in Chapter 13 (Engineering Standards).
 - Foreign keys should be `NOT NULL` and constrained wherever the business relationship is mandatory (e.g., `ledger_entry.lot_id`), directly enforcing NFR-08's "referential and business-rule constraints enforced at the data... layer."
-

Chapter 6 — API Architecture

6.1 Purpose

This chapter specifies the Unified API Layer (PLT-11): its role as the single access point for internal modules, external integrations, and the CRM's read-only stock consumption; its endpoint organization by domain; its contract conventions; and how it enforces the RBAC and audit checkpoints defined in Chapter 4. It expands SRS §3.1, §5 (PLT-11), and the CRM-05 read-only stock requirement.

6.2 Business Context

The API layer is the single place where the business rule "CRM reads, Operations writes" (SRS §3, Chapter 1 §1.4 D2) becomes an enforceable technical contract rather than a convention developers must remember. It's also the layer where the pricing pre-check (CRM-16) and duplicate-detection routing (CRM-21) actually intercept a user action before it completes — making the API layer the practical enforcement point for several business-critical gap-analysis rules, not just a data-access convenience.

6.3 Technical Context

Per SRS §3, the API layer sits between all domain modules and both internal consumers (CRM, dashboards) and external integrations (Oracle, Tally, Coincap, WhatsApp, email, and the newly added GTM Sheet). It is the layer through which the RBAC checkpoint (Chapter 4, §4.4.2) and audit logging (PLT-05) are uniformly applied. **Architect Recommendation:** the API should be organized as a REST resource model grouped by domain (per §6.4.1 below) with a clear read/write asymmetry enforced at the route level for CRM's stock access — CRM-facing stock endpoints should be physically distinct routes (e.g., `GET /stock/*`) with no corresponding write verbs registered, rather than relying on RBAC alone to prevent CRM from writing to the ledger. This gives a second, structural layer of protection for D2 beyond permissions configuration.

6.4 Architecture

6.4.1 API Resource Grouping

Domain	Example Endpoints	Access Pattern
CRM	<code>/leads</code> , <code>/quotations</code> , <code>/win-snapshots</code> , <code>/price-lists</code>	Full CRUD within CRM's own data; <code>/stock/*</code> read-only
Stock (read)	<code>GET /stock/availability/{item_id}</code>	Read-only, consumed by CRM-05
Inventory	<code>/inventory/lots</code> , <code>/inventory/ledger</code> , <code>/inventory/reservations</code>	Write access restricted to Inventory role
Quality Control	<code>/qc/samples</code> , <code>/qc/coa</code> , <code>/qc/batch-release</code>	Write access restricted to QC role
Dispatch	<code>/dispatch/delivery-orders</code> , <code>/dispatch/allocations</code>	Write access restricted to Dispatch role
Shared Platform	<code>/auth</code> , <code>/users</code> , <code>/notifications</code> , <code>/audit</code> , <code>/documents</code> , <code>/dashboards</code> , <code>/reports</code> , <code>/search</code>	Cross-cutting
Integrations	<code>/integrations/oracle/sync</code> , <code>/integrations/tally/post</code> , <code>/integrations/gtm-sheet/sync</code>	System-to-system, service-account authenticated

6.4.2 Request Lifecycle Through the API Layer

```
sequenceDiagram
    participant Client
    participant Gateway as API Gateway (PLT-11)
    participant RBAC as RBAC Check (PLT-02)
    participant Route as Domain Route Handler
    participant Audit as Audit Log (PLT-05)

    Client->>Gateway: HTTPS Request
    Gateway->>Gateway: Authenticate (PLT-01)
    Gateway->>RBAC: Authorize(user, action, resource)
    RBAC-->>Gateway: Allow / Deny
    alt Allowed
        Gateway->>Route: Dispatch to domain handler
        Route->>Route: Execute (may call Automation Engine, Ch.3 §3.4.3)
        Route-->>Gateway: Result
        Gateway->>Audit: Log request + result
        Gateway-->>Client: Response
    else Denied
        Gateway->>Audit: Log denied attempt
        Gateway-->>Client: 403
    end
end
```

6.4.3 CRM-Facing Stock Endpoint — Read-Only Contract (CRM-05)

```
GET /stock/availability/{item_id}?warehouse_id={optional}
```

Response 200:

```
{
  "item_id": "...",
  "available_quantity": 1250.0,
  "unit": "kg",
  "as_of": "2026-08-07T10:15:00Z",
  "fefo_next_expiry": "2026-09-30"
}
```

No `POST`, `PUT`, `PATCH`, or `DELETE` verb exists on any `/stock/*` route. This endpoint reads from the computed-availability view defined in Chapter 5 (§5.4.5), guaranteeing the number shown to a Sales rep quoting (CRM-05) reflects the same ledger state Operations sees — directly serving NFR-05.

6.4.4 Pricing Pre-Check API Contract (CRM-16)

```
POST /quotations/{id}/send
```

Pre-condition check (internal, synchronous):

1. Resolve latest PRICE_LIST_VERSION where status = 'approved' for each line item
2. Compare quotation line rate vs. approved rate within configured tolerance
3. If out of tolerance → 409 Conflict, body includes approved rate + delta
4. If within tolerance → proceed to send, generate PDF (PLT-06), transition quotati

This makes the pre-check a **blocking pre-condition on the send action itself**, not a separate advisory endpoint a Sales rep could bypass by simply not calling it — satisfying CRM-16's intent ("the system verifies the rate... before the quote goes out") as a structural guarantee rather than a UI nudge.

6.4.5 Duplicate Detection API Contract (CRM-21)

POST /leads

Response 200 (created): { "lead_id": "...", "status": "created" }

Response 409 (exact duplicate – hard block):

{ "error": "duplicate_lead", "matched_lead_id": "...", "match_type": "exact" }

Response 202 (near-duplicate – routed for review):

{ "lead_id": "...", "status": "pending_merge_review", "candidate_matches": [...] }

Exact duplicates never create a new row (hard block per CRM-21's specification). Near-duplicates create the lead in a `pending_merge_review` state and notify the Sales Head (via the notification pattern from Chapter 4, §4.4.3) rather than silently creating a fully active duplicate lead.

6.4.6 Integration Endpoint Pattern (INT-05: idempotent, logged, retried safely)

All external-facing integration endpoints (Oracle sync, Tally post, GTM Sheet sync, inbound email webhook) share one contract convention:

Requirement	Mechanism
Idempotency	Each inbound sync/webhook call carries or generates an idempotency key; duplicate keys are detected and return the original result, not a re-processed one
Logging	Every integration call is logged with request, response, and outcome (feeds PLT-05 audit and NFR-09 observability)
Safe retry	Failed calls are queued with exponential backoff; no integration call executes a non-idempotent side effect on retry

6.5 Design Decisions

#	Decision	Rationale

D20	<code>/stock/*</code> routes are structurally read-only — no write verbs registered	Second, structural enforcement of D2 beyond RBAC configuration alone
D21	Pricing pre-check is a blocking pre-condition inside the send endpoint, not a separate advisory call	Prevents a client from bypassing CRM-16 by simply skipping an optional check
D22	Duplicate detection returns distinct HTTP semantics for exact (409, hard block) vs. near-duplicate (202, routed)	Gives API consumers (frontend, integrations) a clear, typed way to handle each case differently, matching CRM-21's differentiated behavior
D23	All integration endpoints share one idempotency/logging/retry contract	Directly implements INT-05 once, uniformly, rather than per-integration

6.6 Advantages

- Structural read-only enforcement (D20) means a future engineer cannot accidentally introduce a CRM-side stock-write endpoint without deliberately breaking an established route-registration convention — a meaningful friction point protecting a core business rule.
- Sharing one idempotency/retry contract (D23, §6.4.6) across five external integrations plus the new GTM Sheet endpoint means INT-05 is implemented and tested once, not five separate times with five chances to get it wrong.

6.7 Disadvantages

- Making the pricing pre-check a synchronous blocking call inside the send path (D21) adds latency to quote-sending and creates a hard dependency on PriceListVersion data being available and performant at send time — a slow or unavailable pricing lookup could block legitimate quote sends. A timeout/fallback policy should be explicitly defined (see Open Questions).
- Distinct HTTP semantics for exact vs. near-duplicate (D22) require frontend and any integration consumers to handle three response shapes for one endpoint (200/409/202) instead of one — added client-side complexity in exchange for API-level clarity.

6.8 Alternatives Considered

Alternative	Why Not Chosen
Enforce CRM read-only-on-stock via RBAC configuration alone	Single point of failure — a misconfigured role could silently violate D2; structural route separation (D20) is defense in depth

Pricing pre-check as an optional, separately-called validation endpoint	Relies on client discipline to call it; doesn't satisfy CRM-16's "before it goes out" as a guarantee
Single response code (e.g., always 409) for both duplicate types	Loses the differentiated hard-block-vs-route-for-review behavior CRM-21 specifically requires

6.9 Dependencies

- §6.4.3 depends on the computed-availability view from Chapter 5 (§5.4.5).
- §6.4.4 depends on PriceListVersion data existing (CRM-14/15), consistent with the sequencing established in Chapter 1.
- §6.4.5 depends on the matching-rule specification flagged as an open question in Chapter 2 (§2.12) — the API contract is defined here, but the actual match logic (exact-match fields, fuzzy-match algorithm) still needs to be specified.
- §6.4.6's idempotency pattern is a prerequisite for GTM Sheet sync (CRM-19) and inbound email capture (CRM-17) functioning safely under retry/duplicate-delivery conditions.

6.10 Future Scope

As integration volume grows, the shared idempotency/retry contract (§6.4.6) could be extracted into a dedicated integration gateway service — **Architect Recommendation:** not needed at current scope; the shared contract implemented within the existing API layer is sufficient through Phase 3, and premature extraction would add operational complexity without a clear current driver.

6.11 Implementation Recommendations

- Define an explicit timeout and fallback behavior for the pricing pre-check (§6.4.4) — e.g., if the PriceListVersion lookup times out, does the send fail closed (blocked) or fail open (warned but allowed)? **Architect Recommendation:** fail closed, consistent with NFR-03's deny-by-default philosophy, with a clear error message rather than a silent block.
- Publish the API contract conventions in §6.4.6 as a shared internal library/middleware so every new integration (including any added post-Phase 3) automatically inherits idempotency, logging, and retry behavior rather than reimplementing it (Architect Recommendation).

6.12 Open Questions

- What tolerance percentage, if any, does the pricing pre-check (§6.4.4) permit before blocking versus warning? Flagged previously in Chapter 3 (§3.12); still unresolved and now blocking a concrete API contract decision.
- What exact fields define an “exact duplicate” versus “near duplicate” lead (§6.4.5)? Flagged in Chapter 2 (§2.12); needed before the matching engine behind this endpoint can be built.
- Should the pricing pre-check failure (409) be overridable by a higher role (e.g., Sales Head can override, rep cannot)? Not specified in either source document.

6.13 Risks

- Synchronous blocking dependency in the quote-send path (§6.7) is a new risk not present in the original SRS risk register; if PriceListVersion lookups become slow under load, it could degrade the core CRM quoting experience — should be added to the consolidated risk register (Chapter 17) with a monitoring/alerting recommendation.
- Undefined duplicate-matching rules (§6.12) risk inconsistent behavior between what engineering builds and what the business actually intends by “exact” vs. “similar.”

6.14 Best Practices

- All API endpoints, including internal-only ones, should pass through the same RBAC/audit checkpoint (§6.4.2) with no exceptions carved out for “trusted” internal callers — consistent with NFR-03’s deny-by-default principle applying uniformly.
- API error responses (e.g., the 409 pricing conflict in §6.4.4) should always include enough structured detail (approved rate, delta) for the calling UI to present an actionable message to the user, not just a generic failure — reduces support burden and aligns with good API design practice generally.

Chapter 7 — Integration Architecture

7.1 Purpose

This chapter specifies the architecture of all external system integrations: Oracle (master data), Tally (valuation), Coincap (price reference), WhatsApp and Email (notifications), and the gap-analysis-introduced integrations — the monitored email inbox (CRM-17, CRM-18), the pricing-team email/upload channel (CRM-14), and the GTM Google Sheet (CRM-19). It expands SRS §10 (Integrations) and §11 (Business Process Automations) where automations cross system boundaries.

7.2 Business Context

Every integration in this platform exists to preserve a business boundary the organization has already decided to keep: Oracle stays the source of truth for master data, Tally stays the system of record for valuation, and the platform does not attempt to become either. The gap-analysis integrations extend this same philosophy into commercial operations — the pricing team's email is treated as an *input* to a governed approval workflow, not an authoritative price source in itself, and the GTM sheet is a *feed* into the CRM pipeline, not a parallel pipeline the business has to maintain by hand. In every case, the business goal is to reduce manual re-keying (Objective O3) without ever letting an ungoverned external input silently become authoritative.

7.3 Technical Context

Per SRS §10, integrations fall into three directional patterns: **read-only inbound** (Oracle), **outbound** (Tally, WhatsApp, Email), and **inbound reference** (Coincap). The gap analysis adds a fourth pattern not present in the original SRS: **inbound unstructured/semi-structured ingestion requiring parsing and human approval** (pricing email, customer email, GTM sheet). All integrations, regardless of pattern, must satisfy INT-05 (idempotent, logged, retried safely on transient failure) — the shared contract already established in Chapter 6 (§6.4.6).

7.4 Architecture

7.4.1 Integration Landscape

```
flowchart LR
    subgraph EXT["External Systems"]
        ORACLE["Oracle"]
        TALLY["Tally"]
        COINCAP["Coincap"]
        WA["WhatsApp"]
        EMAIL_OUT["Email - Outbound"]
        EMAIL_IN["Monitored Inbox - Inbound"]
        PRICE_EMAIL["Pricing Team Email"]
        GTM["GTM Google Sheet"]
    end

    subgraph GATE["Integration Gateway (within API Layer, §6.4.6 contract)"]
        CONN_O["Oracle Connector<br/>INT-01"]
        CONN_T["Tally Connector<br/>INT-02"]
        CONN_C["Coincap Connector<br/>INT-03"]
        CONN_N["Notification Channels<br/>INT-04"]
        CONN_EI["Email Ingestion<br/>CRM-17/18 - NEW"]
        CONN_PL["Price List Ingestion<br/>CRM-14 - NEW"]
        CONN_GTM["GTM Sheet Sync<br/>CRM-19 - NEW"]
    end
```

```

end

subgraph AI_LAYER["AI Services (PLT-10)"]
    PARSE[Entity Extraction /<br/>Classification]
end

subgraph PLATFORM["Platform Core"]
    MASTER[Master Data Cache]
    LEDGER[(Ledger)]
    CRMDB[(CRM Data)]
    PRICING[(Price List Versions)]
end

ORACLE -->|scheduled + on-demand pull| CONN_O --> MASTER
LEDGER -->|valuation events| CONN_T --> TALLY
COINCAP -->|price feed| CONN_C --> PRICING
CONN_N --> WA
CONN_N --> EMAIL_OUT
EMAIL_IN --> CONN_EI --> PARSE --> CRMDB
PRICE_EMAIL --> CONN_PL --> PRICING
GTM --> CONN_GTM --> CRMDB

```

7.4.2 Oracle Connector — Read-Only Master Data (INT-01)

Aspect	Specification
Direction	Inbound, read-only
Trigger	Scheduled (e.g., nightly) + on-demand
Data	Items, partners, reference data
Write behavior	Platform never writes to Oracle
Idempotency	Full-refresh or delta-sync keyed by Oracle record ID; re-running a sync is safe
Failure handling	Per INT-05: logged, retried with backoff; stale master-data cache is preferable to a broken sync blocking the platform

7.4.3 Tally Connector — Outbound Valuation (INT-02)

```

sequenceDiagram
    participant Ledger as Ledger (Ch.5)
    participant Conn as Tally Connector
    participant Tally as Tally

```

```

Ledger->>Conn: Valuation-relevant movement (receipt/issue/adjustment)
Conn->>Conn: Transform to Tally posting format
Conn->>Tally: Post entry (idempotency key = ledger_entry_id)
alt success
    Tally-->>Conn: Ack
    Conn->>Conn: Mark ledger_entry as posted
else failure
    Tally-->>Conn: Error / timeout
    Conn->>Conn: Queue for retry (INT-05)
end

```

Using `ledger_entry_id` as the idempotency key means a retried post can never double-post the same movement to Tally — directly satisfying INT-05 for this specific, financially sensitive integration.

7.4.4 New Integration: Pricing Email/Upload Ingestion (CRM-14)

```

flowchart TB
    A[Pricing Team sends email<br/>with attachment] --> B{Ingestion Method}
    B --> C[Manual Upload]
    B --> D[Email Ingestion Connector<br/>parses attachment format]
    C --> D
    D --> E[Validate structure<br/>item codes, rates, units]
    E --> F[Create PRICE_LIST_VERSION<br/>status = pending]
    E --> G[Reject + notify submitter<br/>with validation errors]
    F --> H[Notify Sales Head<br/>PLT-04, Ch.4 §4.4.3]

```

Architect Recommendation: because sender and format are controlled (internal pricing team, expected template), this connector should use a **deterministic template parser** (fixed CSV/XLSX column mapping) rather than the AI Services layer — this keeps CRM-14/15/16 (the entire pricing-governance workstream) independent of the Phase 3 AI dependency, allowing it to ship in Phase 2 as sequenced in Chapter 1.

7.4.5 New Integration: Customer Email Auto-Capture and Subject Routing (CRM-17, CRM-18)

```

flowchart TB
    A[Monitored Inbox] --> B[Email Ingestion Connector]
    B --> C{Subject/Content Classification<br/>via PLT-10 AI Services}
    C --> D[Create Lead<br/>+ template auto-response]
    C --> E[Extract PO details<br/>match to open Quotation]
    C --> F[Parse into Lead<br/>customer/contact matching]
    D --> G[Confidence Score]
    E --> G
    F --> G
    G --> H[Auto-advance lead stage]
    G --> I[Route to human review queue]

```

D --> J[Lead created via Ch.2 M6 →
Dedup CRM-21 → Routing CRM-20]
F --> J

This is the integration where the AI-layer dependency (flagged repeatedly since Chapter 1) is architecturally unavoidable: unlike pricing emails, customer emails and forwarded correspondence have no fixed template, so deterministic parsing is not viable. All output from this connector — regardless of classification path — terminates at the same Lead creation entry point (M6, established in Chapter 2 §2.5 D8), ensuring dedup and routing logic apply uniformly.

7.4.6 New Integration: GTM Google Sheet Sync (CRM-19)

Aspect	Specification
Direction	Inbound, scheduled or webhook-triggered
Data	Structured rows (unlike email — this is the mechanical case flagged in the original gap-analysis response)
Processing	Row → Lead mapping via fixed column schema, no AI dependency required
Dedup	Routes through the same M6 → CRM-21 dedup path as email-sourced leads
Idempotency	Row identity (e.g., sheet row ID + last-modified timestamp) used as idempotency key to avoid re-importing unchanged rows

Architect Recommendation: consistent with the original gap-analysis response, this integration’s mechanical, structured nature means it does not need to wait for PLT-10 and is a reasonable candidate to pull forward into Phase 2 if business value justifies it — this is a scheduling opportunity, not a requirement change.

7.5 Design Decisions

#	Decision	Rationale
D24	Pricing email/upload ingestion (CRM-14) uses deterministic template parsing, not AI Services	Decouples the entire pricing-governance workstream from the Phase 3 AI dependency
D25	Customer email ingestion (CRM-17/18) uses AI Services (PLT-10) for classification and extraction	No viable deterministic alternative exists for unstructured customer correspondence
D26	All email/sheet-sourced leads terminate at one shared Lead creation entry point	Guarantees dedup (CRM-21) and routing (CRM-20) apply regardless of source, per Chapter 2 D8

D27	Tally posting uses <code>ledger_entry_id</code> as idempotency key	Prevents double-posting to a financial system of record under retry
-----	--	---

7.6 Advantages

- Splitting pricing ingestion (deterministic) from customer-email ingestion (AI-dependent) — D24 vs D25 — means half of the gap analysis's inbound-ingestion scope can ship in Phase 2 without waiting on Phase 3 AI infrastructure, materially de-risking the schedule flagged since Chapter 1.
- A single Lead-creation entry point for all inbound sources (D26) means adding a future fourth or fifth lead source (e.g., a chatbot, a partner portal) is a matter of writing one more connector, not modifying dedup/routing logic again.

7.7 Disadvantages

- The confidence-threshold branch in §7.4.5 (auto-advance vs. human review) is currently unspecified (echoing the open question from Chapter 3 §3.12 and Chapter 4 §4.12) — until resolved, this integration cannot be fully designed at the implementation level, only architected at the pattern level as done here.
- Deterministic template parsing for pricing emails (D24) is brittle to format changes from the pricing team — a process/governance safeguard (e.g., a stable template contract with the pricing team) is needed alongside the technical design, and is outside engineering's control to guarantee alone.

7.8 Alternatives Considered

Alternative	Why Not Chosen
Use AI Services for pricing email parsing too, for consistency	Unnecessarily couples a low-complexity, controlled-format integration to the Phase 3 dependency, for no accuracy benefit given the format is already structured
Build a separate ingestion pipeline for GTM Sheet outside the Lead module	Reintroduces the "three ways in, three behaviors" risk explicitly avoided by D8/D26
Poll Oracle continuously instead of scheduled + on-demand	Unnecessary load on the external system of record for data that changes infrequently; SRS explicitly specifies scheduled + on-demand

7.9 Dependencies

- CRM-14/15/16 (pricing governance) depends only on the deterministic parser (D24) and the Price List Version schema (Chapter 5, §5.4.3) — no PLT-10 dependency, consistent with Phase 2 placement.
- CRM-17/18 depend fully on PLT-10 (AI Services), reinforcing the Phase 3 placement and the concentration risk flagged in Chapter 4 (§4.7, §4.13).
- CRM-19 (GTM Sheet) has no strict AI dependency and could be resequenced into Phase 2, per the Architect Recommendation in §7.4.6 — this should be confirmed with product/business stakeholders, not assumed unilaterally by engineering.
- Tally posting (§7.4.3) depends on ledger entries existing, i.e., depends on Phase 2 Inventory being live.

7.10 Future Scope

Additional inbound channels (e.g., a partner portal, a chatbot-originated lead) would, per D26, integrate as additional connectors terminating at the same Lead creation entry point —

Architect Recommendation: this pattern should be documented as the platform's standard "new lead source" onboarding procedure so future integrations don't require architectural re-litigation.

7.11 Implementation Recommendations

- Confirm with business stakeholders whether CRM-19 (GTM Sheet) should be resequenced into Phase 2 given its low technical complexity relative to its originally-assigned Phase 3 placement (Architect Recommendation, revisiting the original gap-analysis suggestion).
- Establish a template-change notification process with the pricing team so the deterministic parser (D24) is updated proactively rather than failing silently on format drift (Architect Recommendation).

7.12 Open Questions

- What confidence threshold governs CRM-18's auto-advance-vs-review branch? (Recurring open question, now blocking three chapters' worth of implementation detail — this should be prioritized for resolution before Phase 3 design begins.)
- Who owns the pricing-team email template contract, and what is the process if the pricing team changes their export format?

7.13 Risks

- Format drift in the pricing-team email template (§7.7) is a new risk not present in the original SRS risk register; recommend adding to the consolidated risk register (Chapter 17) with a mitigation of automated format-validation alerting (§7.4.4, step E “invalid” branch).
- The unresolved confidence-threshold question, now recurring across four chapters, represents a genuine schedule risk to Phase 3 design readiness if not resolved well ahead of that phase.

7.14 Best Practices

- Every inbound integration should validate and reject malformed input explicitly (as shown in §7.4.4) rather than attempting a best-effort partial import — consistent with the transactional all-or-nothing philosophy established for automations in Chapter 3.
 - Idempotency keys should be chosen from stable, source-provided identifiers wherever possible (Oracle record ID, sheet row ID, `ledger_entry_id`) rather than platform-generated timestamps, to remain correct across retries spanning clock or timezone edge cases.
-

Chapter 8 — Security Architecture

8.1 Purpose

This chapter specifies the security architecture governing authentication, authorization, data protection, audit trail integrity, and secrets management across the platform. It expands SRS §5 (PLT-01, PLT-02, PLT-05), §12 (NFR-01, NFR-03, NFR-10), and addresses the security implications introduced by the gap-analysis integrations (monitored inbox access, GTM Sheet credentials, pricing-team email ingestion).

8.2 Business Context

Security in this platform protects three distinct classes of business value: **commercial data** (customer lists, pricing, win terms — competitively sensitive), **regulated quality data** (COAs, batch release decisions — potentially subject to external regulatory scrutiny per O5), and **financial integrity** (ledger movements that post to Tally). A security failure in any of these is not merely a technical incident — it is a customer trust, regulatory, or financial-reporting incident. This is why NFR-10 (“encryption in transit, least-privilege service accounts, secrets management”) is marked **Must**, not **Should**, in the source SRS.

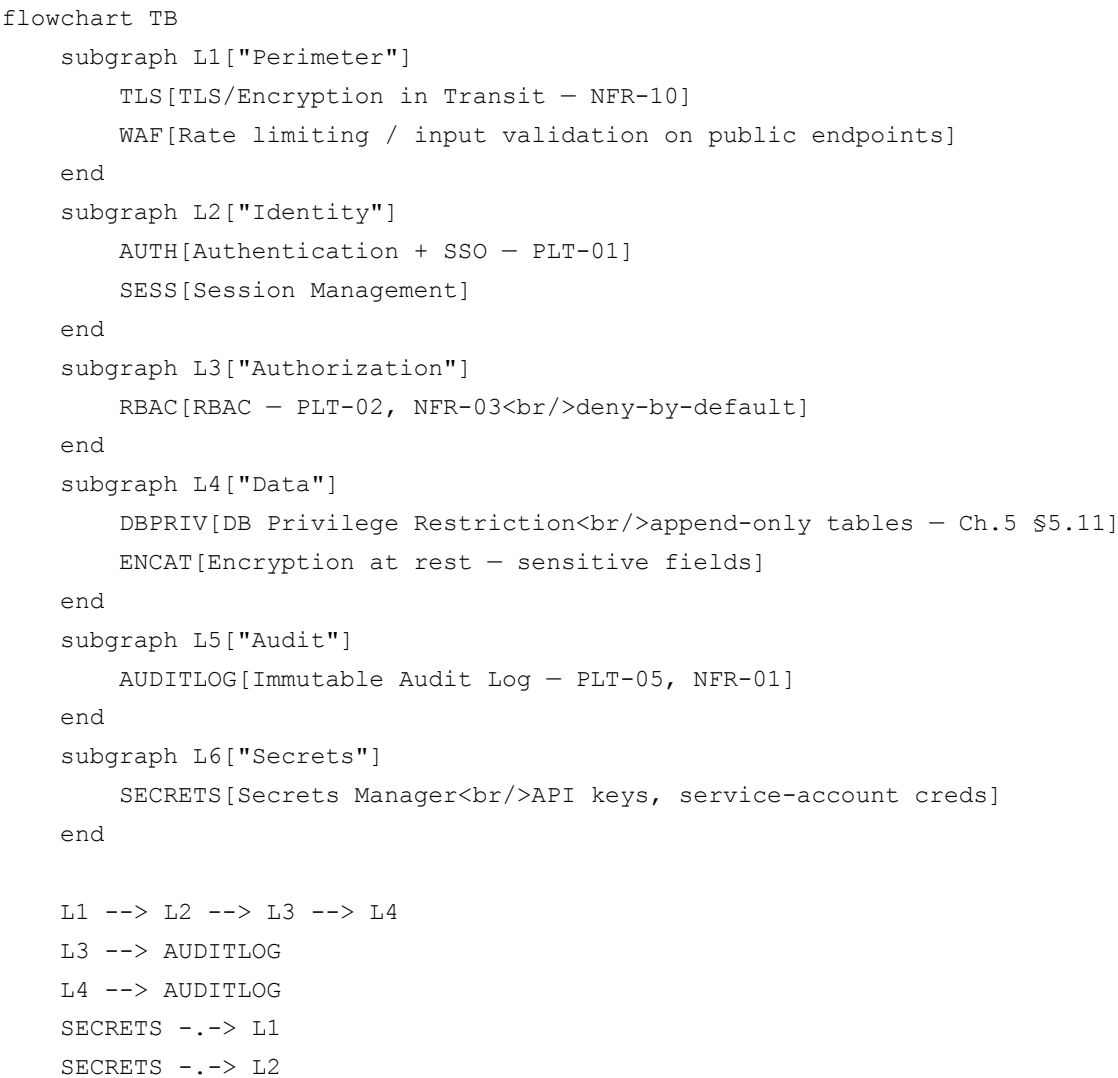
8.3 Technical Context

Security responsibilities are distributed across three points established in earlier chapters:

authentication and RBAC at the API gateway (Chapter 4, §4.4.2; Chapter 6, §6.4.2), database-level privilege restriction on append-only tables (Chapter 5, §5.11), and idempotent, logged, safely-retried integration connectors (Chapter 7). This chapter consolidates those into a coherent security posture and adds the elements not yet covered: encryption, secrets management, and the specific new attack surface introduced by email-based ingestion (CRM-14, CRM-17, CRM-18).

8.4 Architecture

8.4.1 Security Layering



8.4.2 Least-Privilege Service Accounts for Integrations (NFR-10)

Integration	Service Account Scope	Rationale
Oracle	Read-only master data schemas	Never needs write access; matches INT-01's

Connector	only	read-only direction
Tally Connector	Write access to a dedicated posting endpoint/ledger only	Cannot read or modify Tally's other financial data
Coincap Connector	Public/read-only API key, no platform-write scope beyond price reference table	Minimizes blast radius of a compromised external API key
Monitored Inbox (CRM-17/18)	Read-only mailbox access via app-specific credential, no send permission required for capture	A compromised ingestion credential cannot be used to send email as the organization
GTM Sheet Connector	Read-only sheet scope	Platform never writes back to the GTM sheet per CRM-19's stated direction
Pricing Email Connector	Read-only mailbox access, scoped separately from the customer-facing inbox	Isolates pricing-governance ingestion from customer-facing ingestion; a compromise of one does not expose the other

Architect Recommendation: the pricing-team inbox (CRM-14) and the customer-facing monitored inbox (CRM-17) should be **separate mailboxes with separate credentials**, even though both feed the platform — this limits blast radius and keeps the two ingestion pipelines' audit trails cleanly separable, which also aids the traceability goals of AUT-08.

8.4.3 Data Sensitivity Classification

Data Class	Examples	Protection
Regulated/Quality	COA, batch release decisions, quality records	Encrypted at rest; access restricted to QC role + auditors; full audit trail (QC-04 controlled document versioning)
Financial	Win Snapshot terms, ledger valuations, Tally postings	Encrypted at rest; access restricted by role; immutable per Ch.5 design
Commercial/PII	Customer contacts, addresses, lead details	Encrypted at rest; RBAC-gated; subject to standard data-protection handling
Operational	Warehouse/location structure, item master	Standard protection; lower sensitivity
Ingested-Unverified	Parsed emails pending approval, enrichment candidates pending approval (CRM-22)	Isolated in a "pending" state with restricted visibility until approved — never merged into master/authoritative data automatically

8.4.4 Audit Trail Integrity (PLT-05, NFR-01)

Building on Chapter 4's RBAC-checkpoint audit logging and Chapter 5's append-only ledger:

```
flowchart LR
    A[Business Action] --> B[Audit Entry Created<br/>actor, action, entity, before/af]
    B --> C[Storage]
    C --> D[Append-only table<br/>same DB-privilege pattern as Ledger, Ch.5 D16]
    D --> E[No UPDATE/DELETE grant<br/>for application role]
```

Architect Recommendation: the audit log table itself should follow the identical append-only, privilege-restricted pattern established for the ledger in Chapter 5 (§5.11) — an audit trail that can itself be edited is not a meaningful audit trail. This is a direct extension of the “reuse the append-only pattern” principle established since Chapter 1.

8.4.5 Secrets Management

All credentials referenced in §8.4.2 (integration service accounts), plus database credentials and any AI service API keys (PLT-10), are held in a centralized secrets manager, never in source control or application configuration files. Rotation policy and access to the secrets manager itself is governed by the same RBAC/least-privilege principle applied elsewhere in this chapter.

8.5 Design Decisions

#	Decision	Rationale
D28	Separate, scoped credentials per integration (§8.4.2), not one shared service account	Limits blast radius of any single credential compromise; supports AUT-08 attributability
D29	Pricing-team inbox and customer-facing inbox are physically separate mailboxes	Isolates two ingestion pipelines with different trust levels and different downstream consequences
D30	Audit log table follows the same append-only, privilege-restricted DB pattern as the ledger	An editable audit log undermines NFR-01's core guarantee
D31	Ingested-but-unapproved data (CRM-14 pending, CRM-22 enrichment candidates) is stored in a restricted-visibility “pending” state, never auto-merged into authoritative data	Prevents an unverified external input from silently becoming trusted platform data

8.6 Advantages

- Scoped, per-integration credentials (D28) mean a security incident investigation (enabled by AUT-08 and this chapter's audit design) can immediately narrow which system was the entry point, rather than investigating a shared credential's full blast radius.
- Treating ingested-unapproved data as its own sensitivity class (§8.4.3, D31) closes a subtle but important gap: without this, a malicious or malformed inbound email (CRM-17) or a bad web-search enrichment result (CRM-22) could otherwise flow directly into trusted CRM data.

8.7 Disadvantages

- Separate credentials and mailboxes per integration (D28/D29) increase operational overhead — more credentials to provision, rotate, and monitor — a deliberate trade-off of security posture over operational simplicity that should be resourced accordingly in DevOps planning (Chapter 11).
- Encrypting sensitive data classes at rest (§8.4.3) has performance implications for query patterns (e.g., search, PLT-09) that need to operate over encrypted fields — this should be load-tested, not assumed cost-free.

8.8 Alternatives Considered

Alternative	Why Not Chosen
Single shared service account across all integrations	Simpler to provision, but violates least-privilege (NFR-10) and collapses blast-radius isolation
Auto-merge AI-enriched company data (CRM-22) directly into master records	Explicitly contradicted by the gap analysis's own wording ("flagged for approval") — auto-merge without approval is not what was specified
Encrypt only regulated/financial data, leave commercial/PII data unencrypted for query performance	Under-protects PII relative to general data-protection best practice; not justified by the modest performance gain

8.9 Dependencies

- The audit log design (§8.4.4) depends on the same database privilege mechanism established in Chapter 5 (§5.11) — this chapter extends rather than duplicates that mechanism.
- Ingested-unapproved data handling (D31) depends on the Price List pending state (Chapter 5, §5.4.3) and would require an equivalent pending-state schema addition for CRM-22

enrichment candidates, not yet specified in the source gap analysis (see Open Questions).

- Secrets management (§8.4.5) is a dependency for every integration connector described in Chapter 7.

8.10 Future Scope

As the platform matures, field-level encryption could be extended with more granular key management (e.g., per-tenant or per-region keys) if the business expands into jurisdictions with specific data-residency requirements — **Architect Recommendation:** this is speculative beyond current documented scope and should not be built until a specific regulatory driver is identified.

8.11 Implementation Recommendations

- Provision separate credentials for every integration listed in §8.4.2 as part of initial DevOps setup, not retrofitted after a security review (Architect Recommendation).
- Extend the append-only/pending-state pattern (D31) explicitly to CRM-22's company enrichment candidates when that requirement is implemented in Phase 3 — flag this now so it isn't overlooked when Phase 3 design begins (Architect Recommendation).

8.12 Open Questions

- What is the formal data classification and retention policy for regulated quality data (COAs) — is there an external regulatory body governing retention periods that isn't captured in either source document? This connects to the retention question raised in Chapter 5 (§5.12).
- Does the organization have an existing secrets management standard/tool to align with, or is tool selection open to this project? Not specified.
- What schema should hold CRM-22's "pending enrichment" state — a dedicated table, or a status flag on the Customer entity? Not specified in the gap analysis; needs resolution before Phase 3 design.

8.13 Risks

- Encryption-at-rest performance impact on search (§8.7) is currently unquantified and should be added to the Phase 2/3 test plan.
- Absence of a documented regulatory retention requirement (§8.12) for COAs is a compliance risk if such a requirement exists externally but isn't captured in current scope — recommend explicit confirmation with QC/compliance stakeholders before Phase 2 QC module design finalizes.

8.14 Best Practices

- Apply the principle of least privilege not only to integration service accounts (§8.4.2) but to every internal role defined in RBAC (PLT-02) — role definitions should start from zero permissions and add explicitly, never start broad and restrict.
 - Rotate all integration credentials and secrets on a defined schedule (not just on suspected compromise), and log rotation events to the audit trail for full lifecycle traceability.
-

Chapter 9 — Automation Architecture

9.1 Purpose

This chapter specifies the complete automation architecture: every event-driven automation defined in SRS §11 (AUT-01 through AUT-09), the new automations introduced by the gap analysis (stale-lead flagging, price list routing, duplicate routing), and the unifying execution framework that governs all of them. Chapter 3 (§3.4.3) introduced the transactional execution model; this chapter catalogs every automation that runs through it and specifies each one's trigger, effect, and failure behavior individually.

9.2 Business Context

Automations are the mechanism by which the platform delivers Objective O3 (shorten the quote-to-dispatch cycle, reduce manual re-keying) in practice. Every automation in this chapter replaces a manual handoff that today requires a person to notice a state change and act on it. The business risk of getting automation wrong is twofold: an automation that fires incorrectly can reserve or release stock that shouldn't be (undermining O2), and an automation that fails silently reintroduces the exact manual-reconciliation burden the platform exists to remove.

9.3 Technical Context

All automations execute through the transactional execution model established in Chapter 3 (§3.4.3): event → transactional handler → effect-or-retry-queue, with every handler attributable in the audit log (AUT-08) and no handler permitted to partial-commit (AUT-07). This chapter treats that model as given infrastructure and focuses on cataloging what triggers each automation and what each one does.

9.4 Architecture

9.4.1 Complete Automation Catalog

```
flowchart TB
    subgraph SRC["Trigger Sources"]
        T1[Lead Won]
        T2[QC Batch Release]
        T3[Dispatch Executed]
        T4[Stock Below Threshold]
        T5[Approaching Expiry]
        T6[Key State Change]
        T7["Lead stuck at stage - NEW"]
        T8["Price list submitted - NEW"]
        T9["Duplicate lead detected - NEW"]
    end

    end

    subgraph ENGINE["Automation Engine - Ch.3 §3.4.3"]
        E((Transactional<br/>Handler))
    end

    end

    subgraph EFFECTS["Effects"]
        F1["AUT-01: Reserve stock<br/>(concurrency-safe)"]
        F2["AUT-02: Release to sellable<br/>+ notify Inventory"]
        F3["AUT-03: Reduce stock,<br/>close reservation"]
        F4["AUT-04: Low-stock alert<br/>+ reorder workflow"]
        F5["AUT-05: Expiry alert,<br/>prioritize allocation"]
        F6["AUT-06: Notify<br/>(in-app/email/WhatsApp)"]
        F7["CRM-11: Dashboard flag<br/>+ notify owner/manager"]
        F8["CRM-15: Route to<br/>Sales Head for approval"]
        F9["CRM-21: Route to<br/>Sales Head for merge"]
    end

    end

    T1 --> E --> F1
    T2 --> E --> F2
    T3 --> E --> F3
    T4 --> E --> F4
    T5 --> E --> F5
    T6 --> E --> F6
    T7 --> E --> F7
    T8 --> E --> F8
    T9 --> E --> F9
```

9.4.2 Automation Specification Table

ID	Trigger	Effect	Transactional Scope	Failure Path
AUT-01	Lead Won (via Win Snapshot, Ch.5 §5.4.3)	Reserve matching stock (concurrency-safe, Ch.3	Reservation + Ledger entry + Audit	AUT-09 operator queue

		§3.4.2)		
AUT-02	QC batch release	Transition stock to released/sellable; notify Inventory	Quality Record status + Ledger status flag + Audit	AUT-09 operator queue
AUT-03	Dispatch executed	Reduce stock via ledger entries; close reservation	Ledger entry + Reservation closure + Audit	AUT-09 operator queue
AUT-04	Stock below threshold	Raise low-stock alert; optional reorder workflow	Alert record + Notification + Audit	Notification retry (not ledger-critical)
AUT-05	Approaching expiry	Raise expiry alert; prioritize affected lots for FEFO allocation	Alert record + Notification + Audit	Notification retry
AUT-06	Key state changes	Notify via in-app, email, WhatsApp	Notification dispatch + Audit	Channel-level retry (per §7.14 idempotency practice)
CRM-11	Lead stage-age exceeds configured threshold (scheduled scan, Ch.5 §5.4.4 index)	Dashboard highlight flag + notify owner/manager	Flag update + Notification + Audit	Notification retry
CRM-15	Price list version submitted (Ch.7 §7.4.4)	Route to Sales Head for approval	Status transition to <code>pending</code> + Notification + Audit	AUT-09 operator queue (blocks pricing pipeline until resolved)
CRM-21	Near-duplicate lead detected (Ch.6 §6.4.5)	Route to Sales Head for merge decision	Lead status = <code>pending_merge_review</code> + Notification + Audit	AUT-09 operator queue

9.4.3 Reservation Release on Cancellation (INV-11)

A ninth *system-level* automation not explicitly numbered in SRS §11 but specified in §7.3: **INV-11**

("Reservation release when a source event is cancelled or expires"). This is architected as the compensating counterpart to AUT-01:

```
sequenceDiagram
    participant Src as Source Event<br/>(Lead lost/cancelled, Win Snapshot superseded
    participant Eng as Automation Engine
    participant Inv as Inventory Service
    participant DB as Ledger

    Src->>Eng: Cancellation/expiry event
    Eng->>Inv: Release reservation(reservation_id)
    Inv->>DB: BEGIN TRANSACTION
    Inv->>DB: INSERT ledger entry (type=release, qty=+reserved_qty)
    Inv->>DB: UPDATE reservation status = released
    Inv->>DB: COMMIT
    Inv->>Eng: Confirmed
    Eng->>Eng: Audit entry (AUT-08)
```

Architect Recommendation: INV-11 should be implemented in the same Automation Engine framework as AUT-01-06 and the new gap-analysis automations, not as a special case — it shares the identical transactional and audit requirements (AUT-07/08) even though the SRS numbers it separately in the Inventory section rather than the Automations section.

9.4.4 Failed Automation Operator Queue (AUT-09)

```
flowchart LR
    A[Automation Handler Fails<br/>Transaction Rolled Back] --> B[Operator Queue<br/>]
    B --> C{Operator Reviews}
    C -->|Retry| D[Re-execute Handler]
    C -->|Manual Resolution| E[Manual ledger/state correction<br/>with reason code, p]
    D --> F{Success?}
    F -->|Yes| G[Closed]
    F -->|No| B
```

This queue is shared infrastructure across every automation in §9.4.2 whose failure path is marked "AUT-09 operator queue" — it is not a per-automation feature but a single, central failure-handling surface, consistent with the "build once, reuse" principle applied throughout this ADD.

9.5 Design Decisions

#	Decision	Rationale
D32	INV-11 (reservation release) implemented in the same engine/framework as numbered AUT	Shares identical transactional/audit requirements; no reason for a separate

	automations	implementation path
D33	New gap-analysis automations (CRM-11, CRM-15, CRM-21) treated as first-class automations within the same catalog and engine, not bolt-on features	Keeps AUT-07 (transactional integrity) and AUT-08 (attributability) guarantees uniform across original and new automations
D34	CRM-15's failure path blocks the pricing pipeline (routes to AUT-09) rather than silently allowing a price list to remain stuck	An un-routed pending price list is a business-visible failure (pricing team waiting), so it must surface, not fail silently

9.6 Advantages

- Cataloging all nine automations (six original + INV-11 + three new) in one table (§9.4.2) gives engineering, QA, and product one authoritative reference for what the system does automatically — critical for both test-plan coverage and operator training.
- Treating the gap-analysis automations as first-class members of the same engine (D33) means QA can test AUT-07/08 compliance once, against the framework, rather than re-verifying transactional integrity per automation.

9.7 Disadvantages

- The shared AUT-09 operator queue (§9.4.4), while efficient to build once, becomes a single point of operational load — if multiple automation types fail simultaneously (e.g., during an integration outage), operators need clear triage/prioritization guidance (which failures are ledger-critical vs. merely notification-level) to avoid the queue becoming an undifferentiated backlog.
- CRM-15's blocking failure path (D34) means a stuck price-list approval could, in the worst case, delay every quote pre-check (CRM-16) that depends on a current approved price list — this coupling should be explicitly communicated to the Sales Head role as an SLA expectation, not left implicit.

9.8 Alternatives Considered

Alternative	Why Not Chosen
Separate, purpose-built engines per automation category (ledger automations vs. CRM automations)	Duplicates the transactional/audit framework for no architectural benefit; contradicts the “build once” principle established since Chapter 3
CRM-15 failure fails silently, price list	Contradicts AUT-09's own principle that failures must

simply stays pending indefinitely	surface for operator review
Separate operator queues per automation type	Fragments operator workflow; a single triaged queue with priority/category tagging is simpler to build and use, provided triage guidance (§9.7) is defined

9.9 Dependencies

- Every automation in §9.4.2 depends on the transactional execution model from Chapter 3 (§3.4.3).
- CRM-11's automation depends on the composite index defined in Chapter 5 (§5.4.4).
- CRM-15's automation depends on the Price List Version schema (Chapter 5, §5.4.3) and the notification pattern (Chapter 4, §4.4.3).
- CRM-21's automation depends on the duplicate-detection API contract (Chapter 6, §6.4.5) and its still-unresolved matching-rule specification (open question, recurring since Chapter 2).

9.10 Future Scope

As the automation catalog grows in future phases (e.g., predictive reorder automation building on AUT-04, mentioned speculatively in Chapter 1 §1.10), each new automation should be added to this same catalog and engine — **Architect Recommendation:** maintain §9.4.2 as a living reference table updated with every new automation, so the “one engine, one catalog” discipline doesn't erode as the system grows.

9.11 Implementation Recommendations

- Build operator-queue triage tooling (priority/category tagging per §9.7) as part of the AUT-09 implementation itself, not as a later addition — this is foreseeable operational need, not speculative (Architect Recommendation).
- Define and communicate an internal SLA for Sales Head price-list approval (CRM-15) given its blocking downstream effect on CRM-16 (Architect Recommendation).

9.12 Open Questions

- Should AUT-04's “optional reorder workflow” be automatic (system places a reorder) or advisory (system alerts a human to decide)? SRS wording (“optional”) suggests advisory but doesn't confirm.
- What operator-queue triage/priority scheme should distinguish ledger-critical failures (AUT-

01/02/03) from lower-urgency ones (AUT-04/05/06, CRM-11)? Not specified in source documents; flagged in §9.7 as needing definition.

9.13 Risks

- Operator queue overload during correlated failures (e.g., an integration outage causing multiple automation types to fail simultaneously) is a new risk not present in the original SRS risk register — recommend adding to Chapter 17's consolidated register with a mitigation of automated severity classification.
- CRM-15's blocking coupling to CRM-16 (§9.7) creates a business-process dependency (Sales Head responsiveness) that engineering cannot control — this should be flagged to business stakeholders as a process risk, not purely a technical one.

9.14 Best Practices

- Every automation handler, regardless of business domain, should log its trigger event type, entity IDs involved, and outcome in a structurally identical audit format — enabling cross-automation reporting (e.g., "automation failure rate by type") without bespoke parsing per automation.
 - New automations added in future phases should be required to specify their AUT-09 failure-severity classification at design time, not left to be inferred later — codify this as a Definition of Done item in engineering standards (Chapter 13).
-

Chapter 10 — AI Architecture

10.1 Purpose

This chapter specifies the architecture of the AI Services Layer (PLT-10) and its four consuming requirements: CRM-09 (drafting/summarization, original SRS scope), and the three gap-analysis requirements structurally dependent on it — CRM-17 (email-to-lead parsing), CRM-18 (subject routing and PO extraction), and CRM-22 (company web-search enrichment). It expands SRS §5 (PLT-10, priority Could, Phase 3) and consolidates the AI-dependency concentration flagged repeatedly since Chapter 1.

10.2 Business Context

AI capability in this platform exists to reduce manual effort in two specific places the business has identified: drafting/summarization work for Sales (CRM-09, reducing time spent on routine writing), and converting unstructured inbound communication (customer emails, purchase

orders, company research) into structured CRM data without manual re-keying (CRM-17/18/22 — the same O3 objective driving the automation architecture in Chapter 9). The business should understand this capability as **assistive and approval-gated**, not autonomous — every gap-analysis requirement that touches AI output explicitly routes to human review or approval (CRM-18's confidence-threshold branch, CRM-22's "flagged for approval") rather than auto-committing AI output as trusted data.

10.3 Technical Context

PLT-10 is marked **Could** priority and placed in **Phase 3** in the source SRS — the only priority/phase combination among all eleven PLT services. This chapter treats that placement as fixed (it is the client's documented decision, not an architectural judgment this ADD should override) while making explicit, as done in Chapters 1, 4, and 7, that four requirements across two documents are structurally blocked on it. Chapter 7 (§7.4.4, D24) already established that pricing-email ingestion (CRM-14) does **not** depend on this layer — that decoupling remains in force here and is not revisited.

10.4 Architecture

10.4.1 AI Services Layer — Internal Structure

flowchart TB

```
subgraph CONSUMERS["Consumers"]
    C1["CRM-09: Drafting/Summarization"]
    C2["CRM-17: Email → Lead"]
    C3["CRM-18: Subject Routing + PO Extraction"]
    C4["CRM-22: Company Enrichment"]
end

subgraph PLT10["AI Services Layer (PLT-10)"]
    IFACE["Stable Internal Interface<br/>(Ch.4 §4.11 recommendation)"]
    CLASS["Classification<br/>(enquiry / PO / forwarded email)"]
    EXTRACT["Structured Extraction<br/>(entities + confidence score)"]
    DRAFT["Drafting / Summarization"]
    SEARCH["Web Search Enrichment"]
end

subgraph MODEL["Model Provider Layer"]
    LLM["LLM API<br/>(provider TBD — Ch.6 open question analog)"]
end

subgraph GOVERN["Governance"]
    CONF["Confidence Threshold Gate"]
    AUDIT["Audit Log — PLT-05"]
    PENDING["Pending/Approval State<br/>(Ch.8 §8.4.3, D31)"]
end
```

```

end

C1 --> DRAFT
C2 --> EXTRACT
C3 --> CLASS
C3 --> EXTRACT
C4 --> SEARCH

CLASS --> IFACE
EXTRACT --> IFACE
DRAFT --> IFACE
SEARCH --> IFACE
IFACE --> LLM

EXTRACT --> CONF
SEARCH --> CONF
CONF -->|above threshold| DIRECT[Direct effect]
CONF -->|below threshold| PENDING
IFACE --> AUDIT

```

10.4.2 Structured Extraction Contract (Shared by CRM-17/18)

Per Chapter 4's recommendation (§4.11), the AI layer exposes one stable internal interface rather than bespoke logic per consumer:

```

Interface: extract_structured_entity(text, schema, context)
Returns: {
  extracted_fields: { ... per requested schema ... },
  confidence_score: 0.0-1.0,
  classification: "enquiry" | "purchase_order" | "forwarded_email" | "unclassified",
  raw_source_reference: <email_id>
}

```

Both CRM-17 (parse a forwarded customer email into a lead) and CRM-18 (classify and extract from subject-routed email, including PO matching) consume this same interface with different `schema` arguments — CRM-17 requests a lead schema (customer, contact, enquiry summary); CRM-18 requests a PO schema (PO number, line items, quantities, referenced quotation) or a routing classification only. This directly implements the D25 decision from Chapter 7 (§7.5).

10.4.3 Confidence-Gated Effect Routing

This is the concrete design answering the confidence-threshold open question raised repeatedly since Chapter 3 (§3.12), Chapter 4 (§4.12), and Chapter 7 (§7.12):

```

flowchart LR
    A[extract_structured_entity result] --> B{confidence_score<br/>vs. threshold}

```

```

B -->|>= threshold| C["Auto-apply effect<br/>(create lead / advance stage)"]
B -->|< threshold| D["Route to human review queue<br/>(same pattern as AUT-09, Ch
C --> E[Audit entry: auto-applied, confidence=X]
D --> F[Audit entry: pending review, confidence=X]

```

Architect Recommendation: the confidence threshold itself should be a **configurable value per extraction type** (e.g., PO extraction may warrant a higher threshold than lead classification, given PO extraction directly triggers a stage advance with commercial implications), not a single global constant — and should default conservatively (high threshold, favoring human review) at launch, with the threshold tuned downward over time only as observed accuracy data justifies it. This is a recommendation, not a documented requirement, because neither source document specifies a threshold value or governance process.

10.4.4 Company Enrichment Flow (CRM-22)

```

sequenceDiagram
    participant Trigger as New Company Created
    participant AI as AI Services (Web Search)
    participant Pending as Pending Enrichment State<br/>(Ch.8 §8.12 – schema TBD)
    participant SalesHead as Approver

    Trigger->>AI: Enrich(company_name)
    AI->>AI: Web search + structured summarization
    AI->>Pending: Store candidate enrichment data
    Pending->>SalesHead: Notify (PLT-04 pattern, Ch.4 §4.4.3)
    SalesHead->>Pending: Approve or Reject
    alt Approved
        Pending->>Trigger: Merge into Company record
    else Rejected
        Pending->>Pending: Discard, log rejection reason
    end
end

```

This flow enforces D31 from Chapter 8 (§8.5): enrichment data never auto-merges into authoritative Company records. The “schema TBD” note carries forward the open question raised in Chapter 8 (§8.12) about where pending enrichment state is stored — this chapter does not resolve it, only confirms the behavioral contract around it.

10.4.5 Model Provider Abstraction

Architect Recommendation: the AI Services Layer should sit behind a provider-agnostic interface (§10.4.1, `IFACE`) rather than binding CRM-09/17/18/22 directly to a specific LLM API. Neither source document specifies a model provider; keeping this abstraction means a future provider change, or use of different providers for different tasks (e.g., a cheaper model for classification, a stronger model for drafting), does not require changes to any of the four consuming requirements.

10.5 Design Decisions

#	Decision	Rationale
D35	Single stable extraction interface (<code>extract_structured_entity</code>) shared by CRM-17 and CRM-18	Avoids duplicate AI integration code for materially similar capability, per Ch.4 §4.11
D36	Confidence threshold is configurable per extraction type, defaulting conservatively	No documented threshold exists; conservative default minimizes risk of incorrect auto-advancement (echoes NFR-03 deny-by-default philosophy)
D37	Model provider access abstracted behind an internal interface, not hardcoded to one vendor	Preserves flexibility; neither source document mandates a provider
D38	CRM-22 enrichment always lands in a pending/approval state, never auto-merged	Direct implementation of Ch.8 D31

10.6 Advantages

- The shared extraction interface (D35) means CRM-17 and CRM-18 — two distinct requirements — share one implementation, one test suite, and one place to improve extraction quality over time.
- Provider abstraction (D37) protects the platform from vendor lock-in on a capability area (LLMs) that is evolving unusually quickly; a design that hardcodes a specific provider today risks becoming a rework item well before Phase 3 even completes.

10.7 Disadvantages

- Confidence-gated routing (§10.4.3) means a meaningful fraction of PO extractions may land in human review rather than auto-advancing, particularly with a conservative default threshold (D36) — this reduces the immediate labor-saving benefit CRM-18 is meant to deliver, and should be set as an explicit expectation with stakeholders rather than an implicit disappointment discovered post-launch.
- Provider abstraction (D37) adds a small amount of design and integration overhead compared to calling a single vendor's API directly — a reasonable trade given the volatility of the LLM provider landscape, but worth acknowledging as non-zero cost.

10.8 Alternatives Considered

Alternative	Why Not Chosen
Separate, bespoke extraction logic for CRM-17 vs CRM-18	Duplicates effort for two structurally similar problems (unstructured text → structured fields)
No confidence threshold — always auto-apply AI extraction results	Contradicts the gap analysis’s own language (“flagged for approval,” implied review for low-confidence PO matches) and NFR-03’s deny-by-default philosophy
Hardcode to a single LLM provider now	Creates avoidable vendor lock-in risk for a fast-moving technology area with no documented provider mandate

10.9 Dependencies

- All four consumers (§10.4.1) depend on PLT-10 existing, and per SRS §14 this is Phase 3 — this remains the single largest scheduling dependency identified across this entire ADD (first flagged Chapter 1 §1.3, recurring through Chapters 4, 7, and this chapter).
- CRM-22’s full implementation additionally depends on the pending-enrichment-state schema question raised in Chapter 8 (§8.12), still unresolved.
- The confidence threshold governance (D36) is a dependency for CRM-18’s auto-advance behavior and should be resolved (per Chapter 9 §9.12’s recommendation) before Phase 3 design begins.

10.10 Future Scope

Beyond the four documented consumers, the same extraction/classification interface (D35) is well-positioned to support future unstructured-data use cases the business may identify (e.g., contract review, additional document types) — **Architect Recommendation:** any such expansion should go through a new SRS revision, consistent with the position taken in Chapter 4 (§4.10) against silently expanding PLT-10’s scope beyond what’s documented.

10.11 Implementation Recommendations

- Build the stable extraction interface (§10.4.2) and provider abstraction (§10.4.5) early in Phase 3 planning, even before CRM-17/18/22’s individual business logic is finalized, so all three consumers build against a real interface rather than a placeholder (Architect Recommendation, reinforcing Ch.4 §4.11).
- Instrument confidence-score distributions in production from day one, so the conservative default threshold (D36) can be data-drivenly tuned rather than adjusted on guesswork (Architect Recommendation).

10.12 Open Questions

- What model provider(s) will be used, and is there an existing organizational vendor relationship or procurement constraint? Not specified in either source document.
- What confidence threshold value should apply per extraction type at launch? Flagged repeatedly (Ch.3, Ch.4, Ch.7, Ch.9); this chapter proposes a governance *mechanism* (D36) but not a specific *value*, which requires business input.
- Should CRM-09's drafting/summarization outputs (used directly by Sales reps in customer communication) carry any confidence-gating or review step, or are they inherently human-reviewed by virtue of a person sending the draft? Not specified.

10.13 Risks

- The Phase 3 concentration risk on PLT-10, tracked since Chapter 1, is formally consolidated here as the single highest-impact scheduling risk in this architecture — it affects four requirements across two source documents and has recurred as an open question in five prior chapters.
- Confidence-threshold miscalibration (too permissive: incorrect auto-advances; too conservative: no labor savings) is a tuning risk that should be explicitly monitored post-launch, not assumed correct at initial configuration.

10.14 Best Practices

- Every AI-generated output that feeds a business effect (lead creation, stage advancement, enrichment) must carry its confidence score and source reference into the audit log (§10.4.1, `AUDIT`), so any downstream dispute about “why did the system do this” is answerable from the audit trail alone — consistent with AUT-08's attributability principle applied to AI-originated automations.
- Treat AI model outputs as untrusted input requiring validation against the target schema before any effect is applied — the same input-validation discipline applied to the pricing email connector in Chapter 7 (§7.4.4) should apply here, regardless of the AI's own confidence score.

Chapter 11 — Deployment Architecture

11.1 Purpose

This chapter specifies the deployment architecture: environment topology, CI/CD pipeline,

infrastructure supporting the shared PostgreSQL datastore (PLT-12) and API layer (PLT-11), backup/recovery mechanics for NFR-07, observability infrastructure for NFR-09, and scalability provisioning for NFR-06. It translates the preceding architectural chapters into an operable production system.

11.2 Business Context

Deployment architecture is where the platform's non-functional promises to the business become operational reality: NFR-07 (availability and recovery) is the difference between a warehouse outage lasting minutes versus a business-day; NFR-06 (scalability) determines whether the system remains usable as the company grows its transaction volume; and NFR-09 (observability) is what allows engineering to detect and resolve the automation and integration failures cataloged in Chapter 9 before they become business-visible incidents. None of the guarantees promised in Chapters 1–10 mean anything to the business if the deployed system cannot stay up, recover from failure, or be monitored.

11.3 Technical Context

The SRS specifies PostgreSQL as the shared transactional datastore with backup and point-in-time recovery (PLT-12) and marks horizontal scalability (NFR-06) and availability/recovery (NFR-07) as **Should**, and observability (NFR-09) also as **Should** — none are **Must**, but all three are load-bearing for the platform's credibility as an enterprise system. **Architect Recommendation:** despite their "Should" priority in the SRS, this ADD recommends treating NFR-06, NFR-07, and NFR-09 as **Phase 1 architectural concerns** even if their full implementation is phased — retrofitting observability or backup strategy onto a live production ledger system is materially riskier than designing for them from the start.

11.4 Architecture

11.4.1 Environment Topology

```
flowchart TB
    subgraph DEV["Development"]
        DEVAPP[App Instances]
        DEVDB[(Dev PostgreSQL)]
    end
    subgraph STAGE["Staging"]
        STAGEAPP[App Instances]
        STAGEDB[(Staging PostgreSQL<br/>anonymized prod-like data)]
        STAGEINT[Integration Sandboxes<br/>Oracle/Tally test endpoints]
    end
    subgraph PROD["Production"]
        LB[Load Balancer]
```

```

APP1[App Instance 1]
APP2[App Instance 2]
APPN[App Instance N]
PRODDB[(Primary PostgreSQL)]
REPLICA[(Read Replica)]
SECRETS[Secrets Manager – Ch.8 §8.4.5]
end

LB --> APP1
LB --> APP2
LB --> APPN
APP1 --> PRODDB
APP2 --> PRODDB
APPN --> PRODDB
PRODDB -.->|streaming replication| REPLICA
APP1 -.->|read-heavy queries e.g. reporting, PLT-08| REPLICA
SECRETS -.-> APP1
SECRETS -.-> APP2
SECRETS -.-> APPN

```

11.4.2 Deployment Pipeline (CI/CD)

```

flowchart LR
    A[Commit] --> B[Automated Tests<br/>unit + integration]
    B --> C{Pass?}
    C -->|No| A
    C -->|Yes| D[Build Artifact]
    D --> E[Deploy to Staging]
    E --> F[Integration Sandbox Tests<br/>Oracle/Tally/Coincap mocks]
    F --> G[Manual Approval Gate]
    G -->|Approved| H[Deploy to Production<br/>rolling/blue-green]
    G -->|Rejected| A
    H --> I[Post-deploy smoke tests]
    I --> J[Audit log: deployment event]

```

Architect Recommendation: given the financial and regulatory sensitivity of the ledger (Ch.5), QC (COA/batch release), and Tally-posting (Ch.7 §7.4.3) code paths, any deployment touching these modules should require a manual approval gate (as shown) rather than fully automated deployment — this is a deliberate trade of deployment velocity for control on the platform’s highest-risk components, and can be relaxed for lower-risk modules (e.g., dashboard UI changes) once the team has established a track record.

11.4.3 Backup and Point-in-Time Recovery (NFR-07, PLT-12)

Aspect	Specification

Full backups	Scheduled (e.g., daily), stored encrypted, off-instance
Point-in-time recovery	Continuous WAL (Write-Ahead Log) archiving, enabling recovery to any point within the retention window
Retention window	To be defined with business/compliance input (open question, echoes retention questions in Ch.5 §5.12, Ch.8 §8.12)
Recovery testing	Architect Recommendation: scheduled recovery drills (e.g., quarterly), not just backup-taking — an untested backup is not a verified recovery capability
Replica role	Read replica (§11.4.1) serves reporting/dashboard read load (PLT-07/08) and can be promoted in a failover scenario

11.4.4 Observability Stack (NFR-09)

flowchart TB

```

APP[Application Instances] --> LOGS[Structured Logging]
APP --> METRICS[Metrics<br/>RBAC permit/deny — Ch.4 §4.11<br/>automation success/]
APP --> TRACES[Distributed Tracing<br/>across API → module → automation engine]

LOGS --> AGG[Log Aggregation]
METRICS --> DASH_OBS[Ops Dashboard]
TRACES --> DASH_OBS

AGG --> ALERT[Alerting]
DASH_OBS --> ALERT
ALERT --> ONCALL[On-call Notification]
```

This stack directly instruments the metrics this ADD has flagged as needed throughout: RBAC permit/deny rates (Chapter 4, §4.11), automation success/failure rates per type (Chapter 9, §9.14), pricing pre-check latency (Chapter 6, §6.13 risk), confidence-score distributions for AI extraction (Chapter 10, §10.11), and integration connector health (Chapter 7's idempotency/retry contract, §6.4.6).

11.4.5 Scalability Provisioning (NFR-06)

Layer	Scaling Strategy
Application tier	Horizontal — stateless app instances behind load balancer, scale by instance count
Database — reads	Read replica(s) absorb reporting/dashboard/search load, keeping write-path (ledger, reservations) latency stable

Database — writes	Single primary, per modular-monolith decision (Ch.2 D7); scaling writes further is explicitly deferred (Ch.5 §5.10 Future Scope)
Automation Engine	Queue-based (Ch.3 §3.4.3), horizontally scalable consumers processing the automation queue
AI Services (Phase 3)	Provider-abstracted (Ch.10 D37); scaling is largely delegated to the model provider, with the platform's own extraction/classification layer scaled independently

11.5 Design Decisions

#	Decision	Rationale
D39	Manual approval gate for deployments touching ledger, QC, or Tally-posting code paths	Deliberate control-over-velocity trade-off for the platform's highest financial/regulatory-risk components
D40	Read replica dedicated to reporting/dashboard/search load	Protects write-path (reservation) latency from being degraded by read-heavy reporting queries — directly supports NFR-04/NFR-05
D41	NFR-06/07/09 treated as Phase 1 architectural concerns despite "Should" SRS priority	Retrofitting backup, scaling, and observability onto a live ledger system is materially riskier than designing for them upfront
D42	Scheduled recovery drills, not just backup scheduling	An untested backup is not a verified recovery capability; this closes that gap explicitly

11.6 Advantages

- Directing reporting/dashboard read load to a replica (D40) means the reservation transaction's row-level locking (Chapter 3, §3.4.2) — the platform's most concurrency-sensitive operation — is never contending with a heavy PLT-08 report query on the same database connection pool.
- Treating observability as Phase 1 (D41) means the metrics needed to validate Chapter 3's load-testing recommendation (§3.11) and Chapter 6's pricing-pre-check latency concern (§6.13) are actually available when those tests run, rather than being built reactively after an incident.

11.7 Disadvantages

- The manual approval gate for high-risk deployments (D39) slows release velocity for the

modules (Inventory, QC, Dispatch, Tally integration) that are also the most frequently iterated on in Phase 2 — this trade-off should be explicitly accepted by engineering leadership, not discovered as a friction point mid-Phase-2.

- A single-primary write database (§11.4.5), consistent with the modular-monolith decision, remains a long-term scaling ceiling; this chapter operationalizes that known limitation rather than resolving it, consistent with Chapter 5’s Future Scope position.

11.8 Alternatives Considered

Alternative	Why Not Chosen
Fully automated deployment for all modules, including ledger/QC/Tally paths	Prioritizes velocity over control on the platform’s most consequential code paths; not appropriate given NFR-01/02/10’s emphasis on integrity and security
No dedicated read replica; all queries against primary	Risks reporting/dashboard load degrading reservation transaction latency under concurrent use, directly threatening NFR-04/05
Defer observability build-out to Phase 2/3, after core features ship	Contradicts D41’s rationale; makes the load-testing and monitoring recommendations made in Chapters 3, 6, 8, and 10 unimplementable when needed

11.9 Dependencies

- The observability stack (§11.4.4) is a dependency for validating nearly every load/performance recommendation made in this ADD: Chapter 3 §3.11 (reservation load test), Chapter 6 §6.13 (pricing pre-check latency), Chapter 8 §8.13 (encryption performance), Chapter 10 §10.11 (confidence-score tuning).
- Backup/recovery (§11.4.3) depends on a defined retention policy, which is an open question carried since Chapter 5 (§5.12) and Chapter 8 (§8.12) — this chapter cannot resolve it, only note that deployment configuration is blocked on the answer.
- Secrets management (Chapter 8, §8.4.5) is a dependency for secure deployment of every environment in §11.4.1.

11.10 Future Scope

If write-path scaling becomes a genuine constraint (per Chapter 5 §5.10’s future-scope note), deployment architecture would need to evolve toward write-sharding or a service-extraction strategy — **Architect Recommendation:** not addressed further here, consistent with this ADD’s consistent position that speculative future-scale work should wait for evidence from production

load testing.

11.11 Implementation Recommendations

- Stand up the observability stack (§11.4.4) and the read replica (§11.4.1) during Phase 1, ahead of Phase 2's ledger/reservation work going live, so both are proven infrastructure by the time they matter most (Architect Recommendation).
- Schedule the first recovery drill (D42) before Phase 2 production go-live, not after, to validate the backup strategy against real (or realistic staging) data before the ledger is carrying live financial and quality data (Architect Recommendation).

11.12 Open Questions

- What backup retention window satisfies both operational recovery needs and any external compliance requirement — still unresolved from Chapter 5 (§5.12) and Chapter 8 (§8.12), now also blocking concrete backup configuration in this chapter.
- What is the target RTO (Recovery Time Objective) and RPO (Recovery Point Objective) for the platform? Not specified in either source document; needed to size the backup/replication strategy precisely.
- Is there an existing organizational cloud/infrastructure provider standard this deployment should align with, or is provider choice open?

11.13 Risks

- Undefined RTO/RPO (§11.12) is a risk to sizing this chapter's backup and replication strategy correctly — recommend resolving before Phase 2 infrastructure provisioning finalizes.
- Deployment velocity friction from the manual approval gate (§11.7) could create pressure to bypass it under delivery deadline pressure — recommend this be explicitly governed as a non-negotiable control in engineering standards (Chapter 13), not left to case-by-case judgment under pressure.

11.14 Best Practices

- Every production deployment, whether gated or automated, should produce an audit log entry (§11.4.2, step J) — deployment events are business-significant actions under the spirit of NFR-01, even though the SRS's audit requirement (PLT-05) is framed around business data rather than infrastructure events.
- Recovery drills (D42) should test restoration into an isolated environment, never against production, and should be timed and documented to build an evidence trail of actual

Chapter 12 — Engineering Discussion Plan

12.1 Purpose

This chapter consolidates every open question, unresolved dependency, and decision point raised across Chapters 1–11 into a structured discussion plan for the engineering team, product owners, and relevant business stakeholders. Its purpose is to convert this ADD from a static reference into an actionable pre-development agenda — nothing in Chapters 13 onward (Standards, Execution Plan, Timeline, Team Allocation) can be finalized responsibly until the items in this chapter are resolved.

12.2 Business Context

Several of the open questions catalogued here have direct commercial consequence if left unresolved into development: an unresolved pricing-tolerance threshold (Ch.6 §6.12) risks either blocking legitimate quotes or letting bad rates through; an unresolved duplicate-matching rule (Ch.2 §2.12) risks either merging distinct customers or leaving true duplicates unmerged. This chapter exists so these are resolved as deliberate business decisions before engineering encodes an assumption into working software.

12.3 Technical Context

The questions below are organized by discussion session, grouped by the stakeholder group best positioned to answer them, and sequenced by which downstream chapters/decisions they block. Each item references its first appearance in this ADD for full context.

12.4 Architecture (of the Discussion Process)

12.4.1 Discussion Session Plan

flowchart TB

```
S1["Session 1: Business/Product<br/>Commercial rules & thresholds"]
S2["Session 2: Compliance/Legal<br/>Retention & regulatory"]
S3["Session 3: Engineering Leadership<br/>Infrastructure & provider decisions"]
S4["Session 4: Cross-functional<br/>Confirm SRS v1.1 scope & phasing"]
```

```
S1 --> S4
```

```
S2 --> S4
```

S3 --> S4

S4 --> OUT["Output: Finalized SRS v1.1 +
Resolved Assumptions Log"]

12.4.2 Session 1 — Business/Product: Commercial Rules & Thresholds

#	Question	First Raised	Blocks
1	What tolerance (exact match / % deviation) does the pricing pre-check (CRM-16) permit before blocking a quote send?	Ch.3 §3.12	API contract (Ch.6 §6.4.4), backend validation logic
2	Is the pricing pre-check failure overridable by a higher role (e.g., Sales Head)?	Ch.6 §6.12	RBAC design for CRM-16
3	What exact fields define an "exact duplicate" vs. "near duplicate" lead (CRM-21)?	Ch.2 §2.12	Matching engine design (Ch.6 §6.4.5)
4	Should AUT-04's "optional reorder workflow" be automatic or advisory-only?	Ch.9 §9.12	Inventory automation scope
5	What confidence threshold applies per AI extraction type (CRM-18 PO matching vs. CRM-17 lead parsing)?	Ch.3, Ch.4, Ch.7, Ch.9, Ch.10 §10.12	Phase 3 AI design; recurs across 5 chapters
6	Does Win Snapshot support any correction path beyond full supersession?	Ch.3 §3.12	Schema confirmation (Ch.5 §5.4.3, superseded_by)
7	Should CRM-09 drafting/summarization output carry any review/confidence gating?	Ch.10 §10.12	AI governance scope
8	Should Single Sign-On (PLT-01) be mandatory Phase 1 or best-effort?	Ch.1 §1.12	Phase 1 auth implementation

12.4.3 Session 2 — Compliance/Legal: Retention & Regulatory

#	Question	First Raised	Blocks
9	What data retention policy applies to archived Price List versions and superseded Win Snapshots?	Ch.5 §5.12	Schema/purge policy
10	Is there an external regulatory retention requirement for COAs not captured in the SRS?	Ch.8 §8.12	QC module data lifecycle design

11	What is the target RTO/RPO for the platform?	Ch.11 §11.12	Backup/replication sizing
12	What backup retention window satisfies compliance needs?	Ch.5, Ch.8, Ch.11 (recurring)	Deployment configuration (Ch.11 §11.4.3)

12.4.4 Session 3 — Engineering Leadership: Infrastructure & Provider Decisions

#	Question	First Raised	Blocks
13	What LLM/AI model provider(s) will be used — existing vendor relationship or open selection?	Ch.10 §10.12	AI Services Layer build (Ch.10 §10.4.5)
14	Does the organization have an existing secrets management standard to align with?	Ch.8 §8.12	Secrets manager selection (Ch.8 §8.4.5)
15	Is there an existing cloud/infrastructure provider standard?	Ch.11 §11.12	Environment topology (Ch.11 §11.4.1)
16	What schema should hold CRM-22's "pending enrichment" state — dedicated table or status flag?	Ch.8 §8.12	Company enrichment design (Ch.10 §10.4.4)
17	Who owns the pricing-team email template contract, and what's the change-management process?	Ch.7 §7.12	Deterministic parser resilience (Ch.7 §7.4.4)
18	What operator-queue triage/priority scheme distinguishes ledger-critical vs. lower-urgency automation failures?	Ch.9 §9.12	AUT-09 tooling design (Ch.9 §9.11)

12.4.5 Session 4 — Cross-Functional: Scope & Phasing Confirmation

#	Discussion Item	Reference
19	Formal ratification of SRS v1.1 incorporating CRM-11 through CRM-24	Ch.1 §1.11
20	Confirm whether CRM-19 (GTM Sheet) should be resequenced from Phase 3 into Phase 2, given its low technical complexity	Ch.7 §7.11
	Confirm whether CRM-11/20/21 remain in Phase 1 as recommended (pulled	

21	forward from their originally looser prioritization)	Ch.1 §1.14
22	Acknowledge and formally accept the PLT-10/Phase 3 concentration risk affecting CRM-17/18/22	Ch.10 §10.13
23	Accept the deployment velocity trade-off of the manual approval gate on ledger/QC/Tally code paths	Ch.11 §11.13

12.5 Design Decisions

#	Decision	Rationale
D43	Discussion questions grouped by stakeholder authority (business, compliance, engineering leadership) rather than by ADD chapter	Ensures each question reaches the person actually able to answer it, rather than being raised generically to “the team”
D44	Session 4 (cross-functional) scheduled last, after Sessions 1–3	Scope/phasing decisions depend on inputs from all three preceding sessions; sequencing prevents re-litigating decisions made without full information

12.6 Advantages

- Consolidating 23 items scattered across 11 chapters into one prioritized, owner-assigned list prevents any single open question from being silently forgotten between chapters — a real risk given how many recurred without resolution (the confidence-threshold question alone appears in five chapters).
- Grouping by stakeholder authority (D43) means Session 1 doesn’t stall waiting for someone who can only answer a Session 3 question, and vice versa.

12.7 Disadvantages

- Four separate sessions represent real scheduling overhead before development can proceed with full confidence; some engineering teams may be tempted to proceed with default assumptions instead — this ADD’s Architect Recommendations throughout provide reasonable defaults, but they are recommendations, not substitutes for actual stakeholder decisions on business-consequential items (especially #1, #3, #5).

12.8 Alternatives Considered

--	--

Alternative	Why Not Chosen
Resolve all open questions via engineering judgment alone, without dedicated sessions	Several questions (pricing tolerance, duplicate-match fields, retention policy) are business/compliance decisions engineering should not make unilaterally
One single combined session covering all 23 items	Risks the session running long and losing the right stakeholders' attention on questions outside their expertise; splitting by authority (D43) is more efficient

12.9 Dependencies

- Chapters 13 (Engineering Standards), 14 (Micro Execution Plan), 15 (Timeline), and 16 (Team Allocation) all depend on the outcomes of this discussion plan, particularly items #19–23 (Session 4), which directly determine phase scope and sequencing.

12.10 Future Scope

Not applicable — this chapter is itself the mechanism for resolving forward-looking questions raised elsewhere in the ADD; no additional future scope beyond what is already captured in the sessions above.

12.11 Implementation Recommendations

- Schedule Sessions 1–3 within the same one-to-two week window to avoid one delayed session blocking Session 4 indefinitely (Architect Recommendation).
- Produce a written “Resolved Assumptions Log” as the output of Session 4 (§12.4.1) and attach it as a living appendix to this ADD, so resolved decisions are traceable back to the open questions that prompted them (Architect Recommendation).

12.12 Open Questions

All open questions are enumerated in §12.4.2–12.4.4 above; this chapter’s own open question is procedural: **who has authority to make a final call if Session 1–3 stakeholders disagree on a cross-cutting item (e.g., pricing tolerance affects both Sales and Finance)?** This escalation path is not defined in either source document and should be established before Session 4.

12.13 Risks

- If Sessions 1–3 are not completed before Phase 1/2 development begins in earnest, engineering will necessarily encode the Architect Recommendations in this ADD as de facto decisions — acceptable as a stopgap, but each such case should be explicitly logged so it

can be revisited once real stakeholder input arrives, rather than becoming permanent-by-default.

12.14 Best Practices

- Every resolved question from this discussion plan should be written back into the relevant chapter of this ADD as an update (e.g., Chapter 6's pricing tolerance becomes a stated value, not an open question) — keeping the ADD synchronized with actual decisions, consistent with the best-practice stated in Chapter 2 (§2.14) about documentation not drifting from reality.
-

Chapter 13 — Engineering Standards

13.1 Purpose

This chapter codifies the recurring architectural patterns established across Chapters 1–11 into explicit, enforceable engineering standards. Its purpose is to ensure that every engineer joining the project — whether in Phase 1 or Phase 3 — builds new functionality consistent with the patterns already established, rather than each engineer or module reinventing conventions independently.

13.2 Business Context

Consistency at the engineering-standards level is what allows the business promises made in Chapters 1–11 (immutability, auditability, least-privilege, deny-by-default) to hold true not just for the modules explicitly designed in this ADD, but for every feature built after this document is written. A standard, once documented and enforced, protects the business guarantees even as the team and codebase grow beyond what any single architect can review line-by-line.

13.3 Technical Context

Every standard in this chapter has already appeared as a Design Decision or Architect Recommendation in a prior chapter. This chapter's role is not to introduce new architecture but to **extract, consolidate, and formalize** what has already been decided into a checklist-style reference engineers can apply directly during development and code review.

13.4 Architecture (of the Standards Set)

13.4.1 Standard 1 — Append-Only / Immutable Entity Pattern

Applies to: any entity representing a frozen business fact (ledger entries, Win Snapshot, Price List Version, Audit Log).

Rule	Source
No <code>UPDATE / DELETE</code> grant for the application role at the database privilege level	Ch.5 §5.11 (D16), Ch.8 §8.4.4 (D30)
Corrections are new rows referencing the original via a linking field (<code>source_event_id</code> , <code>superseded_by</code>)	Ch.5 §5.4.2, §5.4.3 (D19)
<code>created_at</code> only — no <code>updated_at</code> column	Ch.5 §5.4.2
Every new append-only entity is reviewed against this checklist before implementation	Ch.5 §5.14

13.4.2 Standard 2 — Transactional Automation Handler Pattern

Applies to: every automation (AUT-01 through AUT-06, INV-11, and all gap-analysis automations CRM-11/15/21).

Rule	Source
Single DB transaction per handler execution — full commit or full rollback, never partial	Ch.3 §3.4.3 (D11), Ch.9 (AUT-07)
Every handler logs trigger event ID and effect ID to the audit trail in one entry	Ch.3 §3.14, Ch.9 §9.14 (AUT-08)
Failed handlers route to the shared AUT-09 operator queue with a defined severity classification	Ch.9 §9.4.4, §9.14
New automations specify their failure-severity classification at design time (Definition of Done item)	Ch.9 §9.14

13.4.3 Standard 3 — RBAC + Audit Checkpoint Pattern

Applies to: every API endpoint, with no exceptions for “trusted” internal callers.

Rule	Source
Deny-by-default; permissions granted explicitly, never assumed	Ch.4 §4.4.2 (NFR-03)
RBAC check occurs at the API layer, not duplicated/reimplemented per module	Ch.4 §4.5 (D13)

Both permitted and denied attempts are audit-logged	Ch.4 §4.4.2, §4.5 (D15)
RBAC role definitions start from zero permissions and add explicitly	Ch.8 §8.14

13.4.4 Standard 4 — Least-Privilege Integration Credentials

Applies to: every external system connector (Oracle, Tally, Coincap, WhatsApp, Email, GTM Sheet, any future integration).

Rule	Source
Separate, scoped service account per integration — never shared credentials	Ch.8 §8.4.2 (D28)
Credential scope matches documented integration direction exactly (e.g., Oracle = read-only)	Ch.8 §8.4.2
All credentials in centralized secrets manager, never in source control or config files	Ch.8 §8.4.5
Scheduled credential rotation, logged to audit trail	Ch.8 §8.14

13.4.5 Standard 5 — Idempotent, Logged, Safely-Retried Integration Contract (INT-05)

Applies to: every inbound/outbound integration endpoint.

Rule	Source
Idempotency key derived from stable, source-provided identifiers (not platform-generated timestamps)	Ch.7 §7.14
Every integration call logged with request, response, and outcome	Ch.6 §6.4.6 (D23)
Failed calls queued with exponential backoff; no non-idempotent side effect on retry	Ch.6 §6.4.6, Ch.7 §7.4.3
Malformed input explicitly rejected, never partially/best-effort imported	Ch.7 §7.14

13.4.6 Standard 6 — Single Lead-Creation Entry Point

Applies to: every current and future lead source (manual entry, email capture, GTM sheet, future partner integrations).

--	--

Rule	Source
All lead-creating code paths terminate at one shared creation function	Ch.2 §2.11 (D8), Ch.7 §7.5 (D26)
Dedup (CRM-21) and routing (CRM-20) logic applied uniformly regardless of source	Ch.2 §2.4.2
New lead sources documented as "new connector → shared entry point," never a parallel pipeline	Ch.7 §7.10

13.4.7 Standard 7 — Read-Only Structural Enforcement

Applies to: any consumer that must never write to data it doesn't own (CRM reading stock, integrations reading Oracle).

Rule	Source
Read-only routes contain no write verbs at the route-registration level, not just RBAC restriction	Ch.6 §6.4.3 (D20)
Structural enforcement is defense-in-depth alongside, not instead of, RBAC	Ch.6 §6.5

13.4.8 Standard 8 — AI Output Governance

Applies to: any feature consuming AI Services Layer (PLT-10) output.

Rule	Source
Every AI-driven effect is confidence-gated; below threshold routes to human review	Ch.10 §10.4.3 (D36)
AI output validated against target schema before any effect is applied, regardless of confidence score	Ch.10 §10.14
AI-generated effects carry confidence score and source reference into the audit log	Ch.10 §10.14
No AI output auto-merges into authoritative master data	Ch.8 §8.5 (D31), Ch.10 §10.4.4 (D38)

13.4.9 Standards Applicability Matrix

```

flowchart TB
    subgraph STD["Engineering Standards"]
        ST1[1: Append-Only]
    end

```

```

    ST2[2: Transactional Automation]
    ST3[3: RBAC + Audit]
    ST4[4: Least-Privilege Creds]
    ST5[5: Idempotent Integration]
    ST6[6: Single Lead Entry]
    ST7[7: Read-Only Enforcement]
    ST8[8: AI Governance]
end
subgraph MOD["Applies To"]
    Ledger[Ledger]
    WinSnap[Win Snapshot]
    PriceList[Price List]
    Automations[All Automations]
    API[Every API Endpoint]
    Integrations[All Integrations]
    Leads[Lead Creation]
    Stock[CRM Stock Reads]
    AI[AI Consumers]
end
ST1 --> Ledger
ST1 --> WinSnap
ST1 --> PriceList
ST2 --> Automations
ST3 --> API
ST4 --> Integrations
ST5 --> Integrations
ST6 --> Leads
ST7 --> Stock
ST8 --> AI

```

13.5 Design Decisions

#	Decision	Rationale
D45	Standards documented as extraction/consolidation of prior chapters' decisions, not new architecture	Keeps this ADD internally consistent; avoids introducing standards not traceable to a rationale already established
D46	Each standard includes an explicit "Applies to" scope	Prevents ambiguity about whether a new feature is subject to a given standard

13.6 Advantages

- A code reviewer or new engineer can check any new feature against this chapter's eight standards as a self-contained checklist, without needing to re-read Chapters 1–11 to recall

why each rule exists (though the source references are preserved for anyone who wants the full rationale).

- Because every standard traces to a specific prior decision, this chapter cannot silently drift from the rest of the ADD — any future update to a standard requires reconciling it with its source chapter.

13.7 Disadvantages

- Eight standards, several with multiple sub-rules, represent real onboarding overhead for new engineers — this should be paired with practical tooling (linting, code review templates, CI checks) rather than relying on documentation alone to be read and remembered.

13.8 Alternatives Considered

Alternative	Why Not Chosen
Leave standards implicit in each chapter, without a consolidated reference	Makes cross-cutting review (e.g., “does this new feature follow our append-only pattern?”) much harder to perform consistently
Write standards independent of specific chapter provenance	Loses traceability; a future disagreement about “why do we do it this way” would have no documented answer

13.9 Dependencies

- This chapter depends entirely on Chapters 1–11 already being finalized — it is a synthesis, not an independent architectural input.
- Chapter 14 (Micro Execution Plan) will reference these standards as Definition-of-Done criteria for individual backlog items.

13.10 Future Scope

As new patterns emerge during Phase 2/3 implementation that prove reusable (e.g., a pattern for handling partial integration failures gracefully), they should be added to this chapter as Standard 9, 10, etc., following the same “Applies to” format — **Architect Recommendation:** treat this chapter as a living document updated at phase boundaries, not a one-time artifact.

13.11 Implementation Recommendations

- Convert Standards 1, 3, 4, and 7 into automated CI checks or linting rules where technically feasible (e.g., a migration linter that flags UPDATE/DELETE grants on designated append-

only tables) (Architect Recommendation).

- Include this chapter's standards explicitly in the code review template/checklist used across Phase 1–3 (Architect Recommendation).

13.12 Open Questions

- Should violations of these standards block a pull request automatically (via CI), or rely on human code review judgment? Not specified in either source document; an engineering-culture decision for the team to make.

13.13 Risks

- Standards documented but not enforced via tooling (§13.7) risk erosion over time, especially under delivery pressure — this is a known failure mode for engineering standards generally and should be explicitly mitigated per the recommendation in §13.11.

13.14 Best Practices

- Revisit this chapter at the start of each phase (Phase 2 kickoff, Phase 3 kickoff) to confirm it still reflects actual practice and to add any newly emerged Standard 9+, keeping the “living document” principle from §13.10 actionable rather than aspirational.

Chapter 14 — Micro Execution Plan

14.1 Purpose

This chapter decomposes the entire scope established in Chapters 1–13 — original SRS requirements plus the twelve gap-analysis requirements — into a granular, sprint-ready backlog. It translates architecture into executable work items, sequenced by the dependency chains identified throughout this ADD (e.g., CRM-16 depends on CRM-14/15; CRM-17/18 depend on PLT-10).

14.2 Business Context

A micro execution plan is what allows the business to see, concretely, when value lands — not just “Phase 2 delivers Inventory,” but which specific capability ships in which sprint. This matters for PPO/stakeholder confidence conversations and for setting realistic expectations against the Phase 3/PLT-10 concentration risk flagged since Chapter 1.

14.3 Technical Context

Work items below are organized as epics (mapped to the modules from Chapter 2) broken into stories, each tagged with its governing Engineering Standard(s) from Chapter 13 as a Definition-of-Done reference, and its upstream dependencies.

14.4 Architecture (of the Execution Breakdown)

14.4.1 Epic Breakdown — Phase 1

Epic	Stories (illustrative, not exhaustive)	Standards (Ch.13)	Depends On
E1: Shared Platform Foundation	Auth+SSO setup; RBAC engine; user/org mgmt; audit log table (append-only); secrets manager integration	Std 1, 3, 4	—
E2: API Layer + DB Foundation	API gateway scaffold; PostgreSQL schema migration framework; read replica provisioning (Ch.11)	Std 3, 7	E1
E3: CRM Core	Customer/lead CRUD; pipeline stages; activities/meetings	Std 6	E1, E2
E4: Lead Routing & Dedup	Rule-based routing (CRM-20); exact/near duplicate detection (CRM-21) — pending Session 1 Q3 resolution (Ch.12)	Std 6	E3
E5: Stale-Lead Flagging	Scheduled scan job; composite index (Ch.5 §5.4.4); dashboard flag + notification (CRM-11)	Std 2, 3	E3, E1 (notifications)
E6: Quotation & Win Capture	Quotation CRUD + PDF gen; Win Snapshot entity (append-only, CRM-12); revenue/tonnage calc (CRM-13)	Std 1, 2	E3, E1 (documents)
E7: Dashboards/Reporting/Search Foundation	PLT-07/08/09 scaffolding	—	E2
E8: Read-Only Stock	/stock/* structurally read-only routes (CRM-05, against	Std 7	E2

Interface (seed data)	seed data until Phase 2 ledger exists)		
-----------------------	--	--	--

14.4.2 Epic Breakdown — Phase 2

Epic	Stories (illustrative)	Standards (Ch.13)	Depends On
E9: Inventory Master + Ledger	Item/warehouse/lot/container schema; append-only ledger table (privilege-restricted); computed availability view (Ch.5 §5.4.5)	Std 1	E1, E2
E10: Reservation Engine	Row-level-locked reservation transaction (Ch.3 §3.4.2); FEFO allocation; INV-11 release-on-cancel	Std 1, 2	E9
E11: Quality Control	Sample lifecycle; spec validation; COA generation (versioned document); batch release (AUT-02)	Std 1, 2	E9
E12: Dispatch	Delivery orders; allocation (blocked by QC status, DSP-03); packing; execution (AUT-03); export docs	Std 2	E10, E11
E13: Automation Engine Core	Shared transactional handler framework (Ch.3 §3.4.3); AUT-09 operator queue with triage tagging	Std 2	E9
E14: AUT-01/02/03 Wiring	Lead-won reservation; QC-release notification; dispatch-executed stock reduction	Std 2	E10, E11, E12, E13
E15: Pricing Governance	Price List Version schema (append-only, pending/approved); deterministic email/upload parser (CRM-14); approval workflow (CRM-15)	Std 1, 5	E1 (notifications)
E16: Pricing Pre-Check	Blocking pre-condition in quote-send path (CRM-16), fail-closed on timeout (Ch.6 §6.11) — pending Session 1 Q1/Q2	Std 3	E15, E6
E17: GTM Sheet Sync	Read-only sheet connector; row→lead mapping; idempotency via row ID (CRM-19) — candidate for pull-forward (Ch.7 §7.11, Session 4 Q20)	Std 5, 6	E4
E18: Tally &			

Oracle Integration Hardening	Idempotent Tally posting (ledger_entry_id key); Oracle scheduled+on-demand sync	Std 4, 5	E9, E1
E19: Target Actuals & Effort Reporting	CRM-23 (actuals from Win Snapshot); CRM-24 (activity-vs-results report)	—	E6, E14
E20: Observability & Backup	Full metrics/logging/tracing stack; backup + recovery drill (Ch.11 §11.4.3/§11.4.4) — pending Session 2 RTO/RPO	—	E1, E2

14.4.3 Epic Breakdown — Phase 3

Epic	Stories (illustrative)	Standards (Ch.13)	Depends On
E21: AI Services Layer Foundation	Provider-abstracted interface (Ch.10 §10.4.5); <code>extract_structured_entity</code> contract — pending Session 3 Q13	Std 8	E1, E2
E22: Email Auto-Capture & Routing	Monitored inbox connector (separate creds, Ch.8 D29); classification (CRM-17); PO extraction/matching + confidence gating (CRM-18) — pending Session 1 Q5	Std 4, 5, 6, 8	E21, E4
E23: Company Enrichment	Web-search enrichment; pending-approval schema (Ch.8 §8.12) — pending Session 3 Q16	Std 8	E21
E24: AI-Assisted Drafting	CRM-09 summarization/drafting features	Std 8 (partial — Session 1 Q7 pending)	E21
E25: Advanced Automation & Analytics	Exception handling, cross-domain analytics per SRS §14 Phase 3 scope	Std 2	E13
E26: Integration Hardening (Coincap, WhatsApp)	Production-hardening of remaining integrations	Std 5	E18

14.4.4 Dependency Flow (Cross-Phase)

flowchart LR

```
E1 --> E2 --> E3 --> E4 --> E6
E3 --> E5
E2 --> E8
E1 --> E9 --> E10 --> E14
E9 --> E11 --> E14
E10 --> E11 --> E12 --> E14
E9 --> E13 --> E14
E1 --> E15 --> E16
E6 --> E16
E4 --> E17
E9 --> E18
E6 --> E19
E14 --> E19
E1 --> E21 --> E22
E4 --> E22
E21 --> E23
E21 --> E24
E13 --> E25
```

14.5 Design Decisions

#	Decision	Rationale
D47	Every epic tagged with governing Engineering Standards (Ch.13) as Definition-of-Done	Prevents standards from being “documentation only” (risk flagged in Ch.13 §13.13)
D48	Items blocked by unresolved Ch.12 discussion questions are explicitly flagged in the plan, not silently assumed	Keeps the plan honest about what is and isn’t ready to execute
D49	E17 (GTM Sheet) placed in Phase 2 epic list with explicit pull-forward note, rather than silently moved from its Phase 3 gap-analysis placement	Preserves the original document’s phasing while surfacing the Chapter 7 recommendation for Session 4 to formally decide

14.6 Advantages

- Every epic’s dependency is explicit and traceable to earlier architectural chapters, so sprint planning (Chapter 15) can sequence work correctly without re-deriving dependencies from scratch.

- Flagging Session-pending items directly in the backlog (D48) prevents a common failure mode: a team starting E16 or E22 against a guessed threshold value that later has to be reworked once Session 1/3 actually resolves it.

14.7 Disadvantages

- A backlog this granular will require active maintenance as Chapter 12's sessions resolve open questions — stories tagged "pending" need a defined process for being un-blocked and re-estimated, or they risk stalling silently.

14.8 Alternatives Considered

Alternative	Why Not Chosen
Single flat backlog without epic/phase grouping	Loses the dependency visibility needed to sequence Phase 2's ledger-dependent work correctly relative to Phase 1
Omit "pending Session X" flags and proceed with default assumptions everywhere	Risks silently converting Architect Recommendations into unreviewed permanent decisions, the exact risk flagged in Ch.12 §12.13

14.9 Dependencies

- This entire chapter depends on Chapter 12's discussion plan being executed in parallel with, or ahead of, the sprints that touch flagged items.
- Chapter 15 (Timeline) and Chapter 16 (Team Allocation) both consume this chapter's epic/story/dependency structure directly.

14.10 Future Scope

As Phase 3 epics (E21–E26) are refined closer to their start date, they should be broken down to the same story-level granularity as Phase 1/2 epics here — **Architect Recommendation:** defer that granular breakdown until closer to Phase 3 kickoff, since AI-layer design details (Chapter 10) are themselves partly dependent on Session 3 provider decisions.

14.11 Implementation Recommendations

- Track "pending Session X" flags as a first-class backlog field (not just a text note), so a query can surface all currently-blocked stories at any time (Architect Recommendation).
- Re-estimate E16 and E22 immediately after Session 1 resolves their respective blocking

questions, rather than carrying forward a pre-resolution estimate (Architect Recommendation).

14.12 Open Questions

- What backlog/project-management tool will host this epic structure, and who owns grooming it as Chapter 12's sessions resolve open items? Not specified in source documents — a team-process decision.

14.13 Risks

- If Chapter 12's sessions are delayed, the number of "pending" flagged stories in Phase 2 (E16) and Phase 3 (E22, E23, E24) will grow, creating schedule pressure to start them anyway on assumptions — this directly connects to the risk already flagged in Ch.12 §12.13 and should be monitored by whoever owns backlog grooming.

14.14 Best Practices

- Review the "pending Session X" tagged items at the start of every sprint planning session until all are cleared, so blocked work is never silently pulled into a sprint without its resolving decision in hand.

Chapter 15 — Three Month Timeline

15.1 Purpose

This chapter sequences the Phase 1 epics from Chapter 14 (§14.4.1) into a three-month (12-week) delivery timeline. It scopes strictly to Phase 1 (Shared Platform + CRM, per SRS §14) because that is the phase for which enough architectural and dependency detail exists to commit to weekly sequencing; Phases 2 and 3 depend on Phase 1 completion and on Chapter 12's unresolved discussion items, and are therefore addressed at a coarser, indicative level only.

15.2 Business Context

A three-month horizon is the natural first commitment window for stakeholders evaluating delivery credibility — it's short enough to hold the team accountable to a concrete date, and long enough to ship the full Phase 1 scope (shared platform + working CRM) that SRS §14 identifies as the first value-bearing milestone. This timeline is also the basis against which any PPO/stakeholder confidence conversation about delivery pace should be measured.

15.3 Technical Context

The timeline below sequences Phase 1’s epics (E1–E8, plus E20’s observability foundation pulled forward per Chapter 11’s D41 recommendation) against their dependency graph from Chapter 14 (§14.4.4). Weeks are indicative sprint boundaries (assuming 2-week sprints, 6 sprints total); actual velocity will vary by team size (Chapter 16).

15.4 Architecture (of the Timeline)

15.4.1 Twelve-Week Gantt View

```
gantt
    title Phase 1 – Shared Platform + CRM (12 Weeks)
    dateFormat YYYY-MM-DD
    axisFormat W%W

    section Foundation
    E1 Shared Platform Foundation      :e1, 2026-08-17, 3w
    E2 API + DB Foundation             :e2, after e1, 2w
    E20 Observability Foundation       :e20, after e1, 3w

    section CRM Core
    E3 CRM Core                       :e3, after e2, 2w
    E4 Lead Routing & Dedup           :e4, after e3, 1w
    E5 Stale-Lead Flagging            :e5, after e3, 1w
    E6 Quotation & Win Capture        :e6, after e3, 2w
    E7 Dashboards/Reporting Foundation :e7, after e2, 2w
    E8 Read-Only Stock Interface      :e8, after e2, 1w

    section Sessions
    Ch12 Sessions 1-3                 :sess, 2026-08-17, 2w
    Ch12 Session 4                    :sess4, after sess, 3d
```

15.4.2 Week-by-Week Breakdown

Weeks	Focus	Epics Active	Milestone
1–3	Shared platform foundation: auth/SSO, RBAC, users, append-only audit log, secrets manager	E1	Auth + RBAC deployable to staging
1–2 (parallel)	Chapter 12 discussion sessions run in parallel with E1, not blocking foundation	Ch.12 Sessions	Resolved Assumptions Log (Ch.12 §12.11) produced by end of

	work	1–3	Week 2
~Week 2	Session 4 (cross-functional) — SRS v1.1 ratification, phasing confirmations	Ch.12 Session 4	SRS v1.1 formally approved
3–4	API gateway scaffold, PostgreSQL migration framework, read replica provisioning	E2	API layer live in staging
3–5 (parallel)	Observability stack stood up per Ch.11 D41 (metrics, logging, tracing)	E20	Baseline dashboards operational before CRM Core work begins
5–6	CRM Core: customer/lead CRUD, pipeline stages, activities	E3	Sales rep can create/manage leads end-to-end
7	Lead routing (CRM-20) + duplicate detection (CRM-21) — if Session 1 Q3 resolved by Week 2 per plan	E4	Rule-based routing + dedup live
7 (parallel)	Stale-lead flagging (CRM-11)	E5	Dashboard flag operational
7–8	Quotation + Win Snapshot (append-only) + revenue/tonnage calc	E6	Quote-to-win flow, immutable Win Snapshot generated
7–8 (parallel)	Dashboards/reporting/search foundation	E7	PLT-07/08/09 scaffolding live
9	Read-only stock interface against seed data (CRM-05)	E8	CRM displays stock (seed data placeholder for Phase 2 ledger)
10–11	Integration testing, staging hardening, UAT prep	—	Staging environment matches production-readiness bar
12	UAT, bug-fix buffer, production deployment (per Ch.11 §11.4.2 pipeline, manual approval gate where applicable)	—	Phase 1 production go-live

15.4.3 Critical Path

flowchart LR

E1[E1: Foundation
Wk 1-3] --> E2[E2: API+DB
Wk 3-4]

```

E2 --> E3[E3: CRM Core<br/>Wk 5-6]
E3 --> E6[E6: Quotation+Win<br/>Wk 7-8]
E6 --> UAT[UAT + Go-Live<br/>Wk 10-12]
E1 --> E20[E20: Observability<br/>Wk 3-5]
E20 -.non-blocking, but required before Phase 2.-> UAT

```

The critical path runs **E1 → E2 → E3 → E6 → UAT** — this is the sequence that determines the 12-week floor. E4, E5, E7, E8, and E20 have slack relative to this path and can absorb minor schedule variance without delaying go-live, provided they complete before Week 10 (integration testing).

15.5 Design Decisions

#	Decision	Rationale
D50	Chapter 12 discussion sessions scheduled in Weeks 1–2, parallel to E1, not sequentially before it	Avoids losing 1–2 weeks of foundation-work time waiting on discussions that don’t block E1’s scope
D51	Observability (E20) pulled into Phase 1 Weeks 3–5, parallel to CRM Core	Matches Chapter 11’s D41 recommendation to treat NFR-09 as a Phase 1 concern, not deferred
D52	E4 (routing/dedup) scheduled Week 7, assuming Session 1 Q3 resolves by Week 2	If Q3 (duplicate-match fields) is not resolved by Week 2, E4 is the first epic to slip — flagged explicitly rather than silently assumed resolved

15.6 Advantages

- Running Chapter 12’s sessions parallel to E1 (D50) means the team isn’t idle waiting on business decisions that don’t block early foundation work — a common and avoidable source of schedule slippage.
- Identifying the critical path explicitly (§15.4.3) tells engineering leadership exactly which four epics must not slip, and which four have absorbable slack — a much more actionable planning tool than a flat list of “12 weeks of work.”

15.7 Disadvantages

- The 12-week estimate assumes Session 1–3 resolve on the Week 1–2 schedule assumed in D50; if they slip, E4 (Week 7) is the first casualty, but downstream epics (E6, UAT) are not directly blocked by that specific slip — this asymmetry should be understood by planning stakeholders so a Session delay isn’t assumed to delay the entire timeline.

- This timeline does not yet reflect actual team size/velocity (Chapter 16); it should be treated as a structurally-correct sequencing, subject to recalibration once team allocation is finalized.

15.8 Alternatives Considered

Alternative	Why Not Chosen
Sequence Chapter 12 sessions fully before any Phase 1 work begins	Adds 1–2 weeks of pure delay for decisions that, per D50's analysis, don't block the earliest foundation epics
Defer observability (E20) to end of Phase 1	Contradicts Ch.11 D41's explicit rationale; would leave Weeks 1–9 unobserved during the highest-uncertainty early build period

15.9 Dependencies

- This timeline depends entirely on Chapter 14's epic/dependency structure (§14.4.4) being accurate; any change there requires re-sequencing this chapter.
- Team size and composition (Chapter 16) will determine whether epics shown as parallel (e.g., E4/E5/E6/E7 in Weeks 7–8) can genuinely run concurrently or need to be serialized against a smaller team.

15.10 Future Scope

Phase 2 and Phase 3 timelines should be produced at this same week-by-week granularity once Phase 1 is underway and actual velocity data exists — **Architect Recommendation:** avoid committing to a detailed Phase 2/3 week-by-week schedule now, since Phase 2's reservation-engine and concurrency work (Ch.3) carries genuine estimation uncertainty that Phase 1 velocity data will help calibrate.

15.11 Implementation Recommendations

- Track critical-path epics (E1, E2, E3, E6) with tighter check-in cadence (e.g., weekly) than slack epics (E4, E5, E7, E8, E20), which can be reviewed at normal sprint-boundary cadence (Architect Recommendation).
- Build a lightweight dependency-tracking view (even a simple shared board) that flags in real time whether Session 1–3 outputs have landed in time for E4/E16/E22 as planned (Architect Recommendation, operationalizing D48 from Chapter 14).

15.12 Open Questions

- What sprint length and cadence does the engineering team actually use — this chapter assumes 2-week sprints for illustrative Gantt purposes, but should be confirmed against actual team practice.
- Is Week 12's production go-live subject to any external constraint (e.g., a fixed stakeholder demo date, a fiscal quarter boundary) that should be worked backward from rather than forward toward?

15.13 Risks

- E4's dependency on Session 1 Q3 (§15.5 D52) is the timeline's most concrete near-term schedule risk — recommend explicit tracking as called out in §15.11.
- The 12-week estimate has not yet been validated against Chapter 16's team allocation; presenting it to stakeholders before that reconciliation risks setting an unconfirmed expectation.

15.14 Best Practices

- Revisit this timeline at the end of each 2-week sprint against actual completion, and explicitly re-forecast rather than silently absorbing schedule variance into "we'll catch up later" — consistent with this ADD's general preference for explicit, tracked assumptions over implicit ones.

Chapter 16 — Team Allocation

16.1 Purpose

This chapter proposes a team structure and role allocation capable of executing the Phase 1 timeline defined in Chapter 15, and indicates how that structure should evolve for Phase 2 and Phase 3. It translates the epic/dependency structure from Chapter 14 into staffing requirements by skill area.

16.2 Business Context

Team allocation is where architectural ambition meets budget reality. Every design decision in this ADD — the modular monolith (Ch.2 D7), the concurrency-safe reservation engine (Ch.3), the eight engineering standards (Ch.13) — has a staffing cost. This chapter exists so that the 12-week Phase 1 timeline (Ch.15) is evaluated against a concrete, costable team, not an implicit

assumption of unlimited engineering capacity.

16.3 Technical Context

Role allocation below is derived directly from the epic groupings in Chapter 14 (§14.4.1–14.4.3) and the critical-path analysis in Chapter 15 (§15.4.3). **Architect Recommendation:** team sizing here is a structural minimum to execute Phase 1 within 12 weeks without excessive single-person dependency risk — actual sizing should be reconciled with the organization’s real hiring/budget constraints, which are outside this ADD’s scope.

16.4 Architecture (of the Team Structure)

16.4.1 Phase 1 Team Structure

```
flowchart TB
    subgraph LEAD["Leadership"]
        ARCH["Solution Architect / Tech Lead<br/>1 FTE"]
        PM["Product Manager / BA<br/>1 FTE – owns Ch.12 sessions"]
    end

    subgraph BE["Backend Engineering"]
        BE1["Backend Engineer – Platform<br/>E1, E2, E20"]
        BE2["Backend Engineer – CRM<br/>E3, E4, E5, E6"]
    end

    subgraph FE["Frontend Engineering"]
        FE1["Frontend Engineer<br/>E3, E6, E7, E8 UI"]
    end

    subgraph QAX["Quality & DevOps"]
        QA1["QA Engineer<br/>Test plans, Ch.13 standards verification"]
        DEVOPS["DevOps Engineer<br/>E2 infra, E20 observability, Ch.11 pipeline"]
    end

    ARCH --> BE
    ARCH --> FE
    ARCH --> QAX
    PM --> LEAD
```

16.4.2 Phase 1 Role Allocation Table

Role	FTE	Primary Epics	Standards Ownership (Ch.13)

Solution Architect / Tech Lead	1.0	Cross-cutting review, RBAC/audit design (E1), reservation design prep for Phase 2	All 8 standards — final review authority
Product Manager / Business Analyst	1.0	Owns Ch.12 Sessions 1–4; SRS v1.1 ratification; backlog grooming (Ch.14 §14.11)	—
Backend Engineer — Platform	1.0	E1 (Auth/RBAC/Audit), E2 (API/DB), E20 (Observability)	Std 1, 3, 4
Backend Engineer — CRM	1.0	E3 (CRM Core), E4 (Routing/Dedup), E5 (State-lead), E6 (Quotation/Win Snapshot)	Std 1, 2, 6
Frontend Engineer	1.0	CRM UI for E3/E6, dashboard UI for E7, stock display for E8	—
QA Engineer	1.0	Test plans per epic; standards-compliance verification (Ch.13 §13.11 CI checks)	Verification of all 8
DevOps Engineer	0.5–1.0	E2 infra provisioning, E20 observability stack, Ch.11 CI/CD pipeline, secrets manager	Std 4 (secrets)

Total Phase 1: ~6.5–7 FTE

16.4.3 Phase 2 Team Evolution (Indicative)

Per Chapter 15 (§15.10), Phase 2 timeline is not yet week-by-week sequenced, but team shape can be indicated based on Chapter 14's epic structure:

Addition to Phase 1 Team	Rationale
+1 Backend Engineer — Inventory/Ledger specialist	E9/E10 (ledger, reservation engine) is the highest-complexity, highest-risk work in the ADD (Ch.3 §3.7 lock contention risk) and warrants dedicated focus, not shared attention with CRM work
+1 Backend Engineer — QC/Dispatch	E11/E12 — regulated workflow logic (COA, batch release) benefits from focused ownership given NFR-01 auditability stakes
+0.5 FTE Security/Compliance reviewer	Ch.8's regulated-data handling and Ch.12 Session 2's compliance questions warrant dedicated review bandwidth once ledger/QC data goes live

16.4.4 Phase 3 Team Evolution (Indicative)

Addition to Phase 2 Team	Rationale
+1 AI/ML Engineer	Owns E21 (AI Services Layer foundation, provider abstraction — Ch.10 §10.4.5), the highest-concentration-risk work item in the entire ADD (Ch.10 §10.13)
+0.5 FTE Data/Prompt Engineer	Confidence-threshold tuning (Ch.10 D36), extraction-quality iteration for E22/E23

16.5 Design Decisions

#	Decision	Rationale
D53	Two backend engineers in Phase 1 split by Platform vs. CRM concern, not by seniority	Matches the epic dependency structure (Ch.14 §14.4.1) — Platform epics (E1/E2/E20) are prerequisites for CRM epics (E3–E8), so this split naturally follows the critical path
D54	One dedicated QA engineer with explicit standards-verification responsibility, not QA-as-afterthought	Chapter 13 (§13.7, §13.13) flagged standards erosion as a real risk without tooling/enforcement; a QA owner is a direct mitigation
D55	Product Manager/BA explicitly owns Chapter 12's discussion sessions	These sessions require business/compliance/stakeholder coordination that is a PM/BA function, not an engineering one
D56	Phase 2 adds ledger and QC/Dispatch specialists rather than expanding generalist backend headcount	Reflects Chapter 3's assessment that reservation-engine concurrency work is the platform's single highest-complexity component (§3.7, §3.13) and deserves focused ownership

16.6 Advantages

- Splitting backend engineers by Platform vs. CRM (D53) directly mirrors the critical path identified in Chapter 15 (§15.4.3: E1→E2→E3→E6), meaning the team structure and the schedule structure reinforce rather than fight each other.
- Assigning explicit standards-ownership to the QA role (D54) gives Chapter 13's standards a concrete enforcement owner from day one, rather than leaving enforcement diffuse across the whole team.

16.7 Disadvantages

- At ~6.5–7 FTE for Phase 1, single points of failure exist for several critical-path items (one CRM backend engineer owns E3, E4, E5, and E6) — if that engineer is unavailable, the critical path (Ch.15 §15.4.3) is directly at risk. This should be explicitly flagged to leadership as a staffing risk, not absorbed silently.
- The 0.5–1.0 FTE DevOps allocation may be undersized given the scope assigned to it (infra provisioning, full observability stack, CI/CD pipeline with manual approval gates, secrets manager) — this is flagged as a risk in §16.13 rather than resolved unilaterally, since actual sizing depends on the organization’s existing infra maturity.

16.8 Alternatives Considered

Alternative	Why Not Chosen
Single generalist backend engineer for all of Phase 1	Fails to parallelize Platform (E1/E2) and CRM (E3+) work, extending the 12-week critical path significantly
Fold QA responsibilities into backend engineers’ own testing, no dedicated QA role	Given Ch.13’s explicit standards-erosion risk, a dedicated verification owner is a deliberate mitigation, not a nice-to-have
Bring AI/ML engineer on in Phase 1 to “get ahead”	PLT-10 work has no meaningful starting point until Phase 2/3 architecture (Ch.10) and Session 3 provider decisions (Ch.12) are resolved — early hire would be underutilized

16.9 Dependencies

- This chapter’s allocation depends on Chapter 15’s epic sequencing being accurate; any timeline change should trigger a re-check of whether the proposed team shape still matches the critical path.
- The Product Manager/BA role (D55) is a dependency for Chapter 12’s sessions actually happening on the Week 1–2 schedule assumed in Chapter 15 (§15.4.2, D50).

16.10 Future Scope

Phase 2 and Phase 3 team shapes (§16.4.3, §16.4.4) are indicative and should be firmed up once their respective detailed timelines are produced (per Ch.15 §15.10’s recommendation to defer detailed Phase 2/3 scheduling until Phase 1 velocity data exists) — **Architect Recommendation:** revisit this chapter’s Phase 2/3 sections at each phase kickoff rather than treating them as fixed now.

16.11 Implementation Recommendations

- Cross-train the Platform and CRM backend engineers on each other's epics during Phase 1 to reduce the single-point-of-failure risk flagged in §16.7 (Architect Recommendation).
- Confirm actual DevOps capacity/maturity with engineering leadership before committing to the 0.5–1.0 FTE allocation — if existing infra tooling is immature, this allocation likely needs to increase (Architect Recommendation).

16.12 Open Questions

- Are any of these roles expected to be filled by existing organizational staff versus new hires? Not specified — this ADD proposes role shape, not a staffing/hiring decision.
- What is the actual current DevOps/infrastructure maturity of the organization, which would confirm or challenge the 0.5–1.0 FTE assumption (§16.7)?

16.13 Risks

- Single-point-of-failure on the CRM backend engineer role (§16.7) is a concrete staffing risk directly threatening the Chapter 15 critical path — recommend explicit mitigation planning (cross-training, or increasing to 1.5 FTE on CRM epics) before Week 5 (when E3 begins).
- Potential DevOps under-resourcing (§16.7) risks delaying E20 (Observability), which Chapter 11 (D41) and Chapter 15 (§15.4.1) both treat as a Phase 1 priority, not a deferrable one.

16.14 Best Practices

- Revisit team allocation at each sprint boundary against actual velocity (consistent with Ch.15 §15.14's recommendation to re-forecast rather than silently absorb variance), and treat this chapter's FTE figures as a starting hypothesis to be validated, not a fixed commitment.

Chapter 17 — Risks

17.1 Purpose

This chapter consolidates every risk identified across Chapters 1–16, plus the original SRS risk register (§15.3), into a single authoritative risk register. Its purpose is to give engineering leadership and business stakeholders one place to review, prioritize, and track mitigation for every risk this ADD has surfaced — rather than requiring anyone to re-read sixteen chapters to reconstruct the full risk picture.

17.2 Business Context

Risk consolidation matters commercially because several risks in this register — reservation race conditions, ledger corruption, the PLT-10/Phase 3 concentration risk — connect directly to the business outcomes (O1–O5) defined in Chapter 1. A risk register that stays scattered across chapters is a risk register nobody actually manages; consolidating it here is what makes it operationally usable by whoever owns program risk going forward.

17.3 Technical Context

Each risk below references its first appearance in this ADD, its category, and its current mitigation status as designed in the relevant chapter. Risks are grouped by category (Technical/Architectural, Schedule/Dependency, Compliance/Governance, Staffing/Operational) rather than by chapter, since several risks recur across multiple chapters.

17.4 Architecture (of the Risk Register)

17.4.1 Consolidated Risk Register

#	Risk	Category	First Raised	Mitigation (Designed)	Status
R1	Reservation race conditions under concurrent load	Technical	SRS §15.3; Ch.3 §3.13	Row-level locking, single transaction (Ch.3 §3.4.2, D10)	Designed; needs load-test validation (Ch.3 §3.11)
R2	Ledger corruption from partial writes	Technical	SRS §15.3	Append-only schema, no UPDATE/DELETE grant (Ch.5 D16), transactional automation handlers (Ch.3 D11)	Designed
R3	Integration outages (Oracle/Tally)	Technical	SRS §15.3	Idempotent, logged, retried connectors (Ch.7 §7.4.2/§7.4.3, INT-05)	Designed
				MoSCoW	

R4	Scope creep across modules	Schedule	SRS §15.3	prioritization, phased delivery (SRS §14); explicit epic/dependency structure (Ch.14)	Managed via process
R5	Stale stock in CRM views	Technical	SRS §15.3	Near-real-time propagation via computed view, not cache (Ch.5 §5.4.5, D17)	Designed
R6	Lock contention on hot lots under peak concurrent reservation load	Technical	Ch.3 §3.13	None yet beyond design; requires load testing	Open — needs Phase 2 test plan item
R7	Undefined compensating-correction process for Win Snapshot post-freeze errors	Governance	Ch.3 §3.13	<code>superseded_by</code> schema field exists (Ch.5 D19); process not defined	Open — Ch.12 Session 1 Q6
R8	AI Services Layer (PLT-10) concentration risk — 4 requirements blocked on one Phase 3 “Could” service	Schedule	Ch.1 §1.13; recurs Ch.4, Ch.7, Ch.9, Ch.10 §10.13	Deterministic parsing decoupled where possible (Ch.7 D24 for CRM-14); stable interface planned (Ch.10 D35/D37)	Partially mitigated; core dependency remains
R9	Encryption-at-rest performance impact on search/query	Technical	Ch.8 §8.13	None yet; needs load testing	Open — needs Phase 2/3 test plan item
R10	Undefined data retention policy (Price List, Win Snapshot, COA)	Compliance	Ch.5 §5.12; Ch.8 §8.12	None; requires Ch.12 Session 2 resolution	Open — Ch.12 Session 2 Q9/Q10
	Undefined				

R11	RTO/RPO for backup/recovery sizing	Compliance/Technical	Ch.11 §11.12	None; requires Ch.12 Session 2 resolution	Open — Ch.12 Session 2 Q11
R12	Synchronous pricing pre-check dependency could degrade quote-send latency under load	Technical	Ch.6 §6.13	Fail-closed timeout policy recommended (Ch.6 §6.11)	Partially designed; needs load testing
R13	Undefined duplicate-matching rules (exact vs. near)	Governance	Ch.2 §2.12; recurs Ch.6, Ch.7, Ch.9	API contract designed (Ch.6 §6.4.5); matching logic pending	Open — Ch.12 Session 1 Q3
R14	Confidence-threshold miscalibration for AI extraction (CRM-17/18)	Technical/Governance	Ch.9 §9.13; Ch.10 §10.13	Configurable per-type threshold, conservative default (Ch.10 D36)	Designed; requires production tuning
R15	Format drift in pricing-team email template	Operational	Ch.7 §7.13	Explicit validation + rejection on malformed input (Ch.7 §7.4.4)	Partially mitigated; needs template-change process (Ch.12 Session 3 Q17)
R16	Operator queue (AUT-09) overload during correlated automation failures	Operational	Ch.9 §9.13	Shared queue designed; triage/priority scheme not yet defined	Open — Ch.12 Session 3 Q18
R17	CRM-15's blocking coupling to CRM-16 creates a business-process dependency on	Operational	Ch.9 §9.13	None; requires internal SLA definition	Open — Ch.9 §9.11 recommendation

	Sales Head responsiveness				
R18	Deployment velocity friction from manual approval gate on ledger/QC/Tally paths, risk of bypass under pressure	Operational	Ch.11 §11.13	Recommend governing as non-negotiable in engineering standards	Partially mitigated via Ch.13 Std 2
R19	Single-point-of-failure: one CRM backend engineer owns E3/E4/E5/E6	Staffing	Ch.16 §16.13	Cross-training recommended (Ch.16 §16.11)	Open — needs commitment before Week 5
R20	Potential DevOps under-resourcing (0.5–1.0 FTE) risking E20 (Observability) delay	Staffing	Ch.16 §16.13	Needs confirmation of actual infra maturity	Open — Ch.16 §16.12
R21	Chapter 12 discussion sessions delayed, causing “pending” backlog items to accumulate under schedule pressure	Schedule/Governance	Ch.12 §12.13; Ch.14 §14.13	Sessions scheduled parallel to E1 (Ch.15 D50); explicit backlog flagging (Ch.14 D48)	Managed via process, contingent on Session adherence
R22	Standards documented but not enforced via tooling, risking erosion under delivery pressure	Governance	Ch.13 §13.13	CI/linting recommended (Ch.13 §13.11)	Open — needs tooling investment

17.4.2 Risk Heat Map

```

flowchart TB
    subgraph HIGH["High Impact / High Attention Needed"]
        R1H[R1: Reservation races]
        R8H[R8: PLT-10 concentration]
        R10H[R10: Retention policy undefined]
        R19H[R19: CRM backend SPOF]
    end
    end
    subgraph MED["Medium – Needs Resolution Before Relevant Phase"]
        R6M[R6: Lock contention]
        R12M[R12: Pricing pre-check latency]
        R13M[R13: Duplicate-match rules]
        R14M[R14: Confidence miscalibration]
        R20M[R20: DevOps sizing]
    end
    end
    subgraph LOW["Lower – Process-Managed"]
        R4L[R4: Scope creep]
        R21L[R21: Session delay cascade]
        R22L[R22: Standards erosion]
    end
    end

```

17.5 Design Decisions

#	Decision	Rationale
D57	Risks categorized by type (Technical, Schedule, Compliance, Staffing) rather than by originating chapter	Enables risk-owner assignment by functional area (e.g., all Compliance risks go to Ch.12 Session 2 owner)
D58	Every risk explicitly tagged with mitigation status (Designed / Partially Mitigated / Open)	Distinguishes risks already architecturally addressed from those still requiring a decision or investment — prevents false confidence that “documented” means “resolved”

17.6 Advantages

- A single consolidated register (22 items) makes program-level risk review tractable in one sitting, rather than requiring a reviewer to extract risk statements from sixteen separate chapters.
- Explicit mitigation-status tagging (D58) prevents the common failure mode of an architecture document listing risks that sound addressed simply because they were discussed at length — the register makes clear that discussion and design are not the same as resolution.

17.7 Disadvantages

- A 22-item register risks becoming overwhelming without active ownership; without an assigned owner per risk (not specified in source documents — a program-management decision outside this ADD's scope), the register itself can become another document that's read once and not maintained.

17.8 Alternatives Considered

Alternative	Why Not Chosen
Report only "new" risks not already in the SRS's original register	Would omit R1–R5, which remain live and relevant even though first raised in SRS §15.3 — completeness was prioritized over brevity
Rank risks by a single numeric score (probability × impact)	Neither source document provides probability/impact estimates, and inventing them would misrepresent unvalidated assumptions as quantified analysis; the three-tier heat map (§17.4.2) is a more honest representation of current knowledge

17.9 Dependencies

- Resolution of R7, R10, R11, R13, R15, R16, R17 all depend on Chapter 12's discussion sessions.
- R1, R6, R9, R12 depend on load-testing infrastructure being available, which itself depends on Chapter 11's observability stack (E20).
- R19, R20 depend on Chapter 16's team allocation being finalized against real organizational constraints.

17.10 Future Scope

As Phase 2 and Phase 3 designs mature (per the deferred detailed planning noted in Ch.15 §15.10 and Ch.16 §16.10), new risks specific to those phases should be added to this register following the same category/status format — **Architect Recommendation:** treat this chapter, like Chapter 13, as a living document reviewed at each phase boundary.

17.11 Implementation Recommendations

- Assign a named owner to every risk in §17.4.1, not just a category, as the first action following this ADD's delivery (Architect Recommendation) — this is a program-management step outside this document's own scope but essential to the register's ongoing usefulness.

- Review this register explicitly at each Chapter 12 session and at each phase-boundary retrospective, updating status fields rather than letting them go stale (Architect Recommendation).

17.12 Open Questions

- Who owns this risk register going forward — the Solution Architect (Ch.16 role), the Product Manager/BA, or a separate program-management function? Not specified in either source document.

17.13 Risks

This chapter's own meta-risk: a consolidated risk register that is produced once, at the end of an architecture document, but never revisited becomes stale faster than a register that was never centralized in the first place — false confidence from an outdated register can be worse than the fragmented-but-current alternative it replaced.

17.14 Best Practices

- Treat "Open" status items in §17.4.1 as blocking items for their respective phase's exit criteria, not as background notes — a risk marked Open at Phase 2 kickoff that's still Open at Phase 2's midpoint should trigger explicit escalation, not silent carry-forward.

End of consolidated document — Chapters 1 through 17. Chapter 18 (Recommendations) has not yet been generated; append it to this file once produced.